Discovering obfuscated structures using algebraic statistics and algebraic geometry

Awasthi, Vinay
Johns Hopkins Applied Mathematics
Baltimore, MD 21231
vawasth1@jh.edu

December 16, 2025

Abstract

Graph Neural Networks (GNNs) have achieved great success in link prediction, Graph and Node classification, by generating node embeddings and estimating proximity through distance metrics. However, existing approaches primarily assume nodes represent fixed or simple data, and edges are there connecting these nodes to train from, limiting their applicability to *statistical distributions* on a *statistical manifold*, which can only be connected by hypothesis which will be generated by limited observability of events, connections. Methods based on the Fisher–Rao distance address edge connection, partially but fail to scale to large numbers of distributions. Moreover, current PyTorch-based GNN architectures and pooling operations remain highly dependent on specific node attributes, hindering transferability and adaptation across contexts. This paper proposes a novel framework introducing *Arithmetic Unit (AU)-based node representations* to encode distributions, *blockchain-based message passing* to ensure data integrity, and *Contingency Table-driven MCMC* methods to establish initial causation linking graphs. I then apply algebraic blow up to resolve singularities followed by Algebraic Blowdown to dissipate 70% probability mass, isolating "high probability mass" core structure supporting observable outcomes. This algebraic blowup/blowdown flow happens as entropy simulation runs, so that if some structures of high importance appear, only for short duration, they too are captured.

1 Introduction

The entropy flow is the Riemannian gradient flow of the Kullback–Leibler divergence $\mathrm{KL}(p\|\pi)$ on the probability simplex Δ^{n-1}, endowed with the **Fisher–Rao metric** — the unique (up to scaling) monotone metric invariant under Markov morphisms [82]. Explicitly, for probability densities $p, q \in \Delta^{n-1}$, the Fisher–Rao inner product at p is

$$\langle u, v\rangle_p^{\mathrm{FR}} = \sum_{i=1}^{n} \frac{u_i v_i}{p_i} \qquad (u, v \in T_p\Delta^{n-1} = \{x \in \mathbb{R}^n : \sum x_i = 0\}).$$

This metric is isometric to the standard round metric on the positive orthant of the $(n-1)$-sphere via the map

endowing Δ^{n-1} with constant positive sectional curvature $+4$ (or $+1$ under the conventional $1/4$-rescaling of Amari & Nagaoka [83]). Consequently, the geodesic equations of the Fisher–Rao metric are exactly the equations of the **reversible Fokker–Planck dynamics** whose unique stationary distribution is the prior π. The entropy flow we implement is therefore not an ad-hoc diffusion but the **shortest-path (geodesic) motion** on this positively curved probability manifold toward the prior π. When the flow approaches the boundary of the simplex (where the Fisher–Rao metric degenerates to infinite curvature), the Gerstenhaber–BV homological kicks inject precisely the minimal algebraic perturbation required to escape the singularity while preserving the variational structure, thereby restoring the global geodesic character of Bayesian inference on the connectome. GNNs can only detect locally similar embedding and thus lack, global perspective needed to predict, existence of functional connectomes, operating across hierarchical physical graph structure. GNNs have been used to generate product recommendations for customers, in billion node graphs (pinterest), Equivariant GNNs are also used in performing molecular dynamics simulations, to predict material properties [1, 2], by computing impact of interatomic forces using a cut-off distance based, sphere (Potential Energy Surface PES), where angular momentum of various electrons are projected to get force approximations. These models can give highly accurate results at fraction of the costs of software products such as AMBER, Discovery Studio, GROMACS, LAMMPS etc.[3, 6]. All these methods work at small distances as GNNs dilute local information for long distances in graph, due to constant averaging on message data via permutation invariant aggregation functions. This work investigates obfuscated structures, characterized by temporally evolving observables—ranging from delayed physiological responses in mRNA cases, to coupled kinetic–informational dynamics in covert operations. Within the Arithmetic Unit (AU) framework, such phenomena are analyzed through geometric measures, including the Fréchet mean and Fisher information metric on the underlying statistical manifold. Because these observables lack a stable algebraic closure, no ring-like operations are defined, restricting analysis to distance-based representations within the given applicable context, in arithmetic unit [40]. We do this by lifting samples to tangent space, performing analysis and then projecting data back at manifold, discovering main interactions points, as connectomes [41, 42] describing a levi-cevita connection on a statistical manifold (geodesic path of nodes with highest degree).

Now imagine an mRNA vaccine designed to combat five transcription error-based diseases (ranging from Cystic Fibrosis to Thalassemia). This vaccine utilizes a delivery system where different Lipid Nanoparticles (LNPs) carry various genetic material strands (Alleles A to Allele I). To enhance cellular uptake and prevent the immune system from attacking the LNPs, the mRNA nucleotides are modified with pseudouridine (N^1-methyl-pseudouridine-modified mRNA) [8].

Hypothesis - The OS4T LNP-mRNA system exhibits differential therapeutic efficacy across various combinations of Allele A through Allele I. Specifically, the maximum disease response is contingent upon identifying the minimal required subset of alleles and their optimal strand lengths necessary to sufficiently interfere with or suppress the spectrum of copying errors observed in the disease state (see Table 1) [9, 21].

This research presents a highly efficient structure-discovery framework, validated using the ogbl vessel dataset [10], demonstrating that complex system outcomes can be modeled accurately with only 1 percent of the data. The approach centers on resolving interactions through a specialized GNN where entropy flows are used to directly identify the most influential links and structures. Crucially, we do not train a neural network to find embeddings; rather, the process leverages Jacobians to uncover regions of high interaction density (singularities). This leads to the development of truly personalized medicine [22] tailored to specific brain or cellular geometries, offering a powerful tool for understanding any information-exchanging system.

This paper presents a method for hypothesis testing and for identifying key structures (voxels), links, and underlying structures through algebraic statistics and arithmetic universes, which captures transient temporal behaviors of individual nodes or coordinated groups of nodes (emerging irreducible algebraic variety instead of various disconnected sets, on sets of graphs connected via common outcomes of applied functions, i.e. nodes exhibiting +ve sentiments to an event, non-response, just like chemicals responding to reactants) that collectively actuate events leading to known set of outcomes. These graphs change as we change how probabilities will be controlled to flow (shorter lengths, vs higher cross section of vessel etc..)

Today's AI models usually learn from just one class of datasets (i.e. Language Models are training with text sequential data, rejecting arbitrarily connecting temporal graph dynamic data): they train on most of it, test themselves on a slice from the same source, and then make predictions on the small leftover portion they haven't seen. If input geometry changes (context), these models needs to be retrained. By using 1 percent data, one can predict structures responsible for a given outcome with 90 percent accuracy, retraining these structures with different geometries is very fast as we are not looking to match convergence with a loss function, but just entropy flow to settle.

In this paper, uncertainty is addressed within Arithmetic Units (AU) by detecting shifts in entropy through information geometry. Rather than computing the full Fisher-Rao metric, rapid changes alone are sufficient to trigger indicators [23, 24, 25, 28]. Shannon entropy does not allow gradient to rise sharply, we are not computing cross entropy terms used in normal L computation. The structures (links) connected to the nodes that exhibit the lowest absolute local entropy (S) will identify the most stable, essential core links that carry the highest probability mass (i.e. structure needed to trigger given outcome).

The underlying brain structural manifold gives rise to complex, disease-specific functional networks characterized by distributed, highly interconnected voxels. In the context of epilepsy, these critical functional hubs are observed across a wide array of regions, such as the hippocampal subregions (CA1sp, SUB, HPF), amygdalar nuclei (BLA, LA), and deep brain structures (EP/Epithalamus, LZ/Hypothalamic Lateral Zone). Identifying these distributed functional structures is essential for understanding the precise role that each high-probability voxel plays in the overall chain of neural interactions.

This paper addresses these challenges by encoding behaviors and hypotheses that connect observable events to uncover underlying structures in cases such as diseases, or events like epilepsy connecting entropy to voxels.

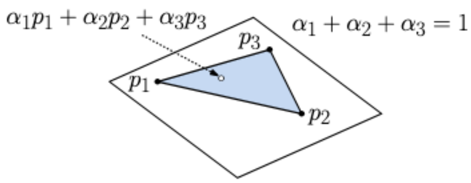
2 Information Geometry, Satistical Manifold of Toric surfaces, Generating Lattice

In this paper we generate data from one table by random walk. It is like finding number of ways a $\mathcal{T}^{nxm}$ table can be filled with non zero positive numbers, while preserving row and column sums. Latte[29] using cddlib[29] solves these Row sum/Column Sum constraints using H-representation using double description method by Motzkin[31]. If we have a system of equations $b - Ax \geq 0$, one does not need to specify all ineqaulities (marginals on side of contingency table), if we also add other constrains limiting (culling regions of very low probabilities, generating million more points). See 4x4MyAlleles input file in project's github leaker repository [51] and cddlib manual Cddlib as input to LattE for my contingency table described below, generating ***150,425,570,4*** for first 3x9 entries. These options implode lattice point computation as numbers go high (larger space), so I will consider D (Spinal Musc. Atrophy) as data with high probability (maximum stable core of probability mass carrying information). We are generating data with conditional independence, iid is not assumed as we are testing hypothesis of various inter-dependences via models.

2.1 Probabilities on Manifold, Simplices and Connection to Algebraic Geometry

When one considers discrete probabilities of 3 events, we get a triangle where lines connecting edges describe probability distribution of say observing heat vs tail, vertices describe p1=HH, p2=TT, p3=HT, as points on face of triangle (convex hull) describe probability of seeing two heads, two tails or head and then tail, with fixed marginal constraints in contingency table (H-representations). If we were to also consider p4=TH, we will then get a tetrahedron 3-Simplex, where lines connecting opposite side (i.e. observing p1p2, p3p4, will form a curved toric surface. Points on this surface will represent cases where p1p2 - p3p4 = 0, point greater than 1, are on the upper side (OR=2.56), point below represent less than 1 case. Simpsons's Paradox occurs when we connect points crossing toric surface. Upper region indicates region of improbable events (2.56 as probability can't be greater than 1). A polytope lattice can't capture such finer details as toric can (segre variety represents functions such as xy, yz coming from 2x2 minors).

Information Geometry of Statistical manifolds (simplices describing probabilities with constraints) allows us to deal with curvatures, get Levi-Cevita connections (paths of point with maximum connectivity) while combining lattice/toric representations of probability spaces to quickly establish set of key hypotheses of high value (strong correlation), where multiple actors can be forming deep structures.



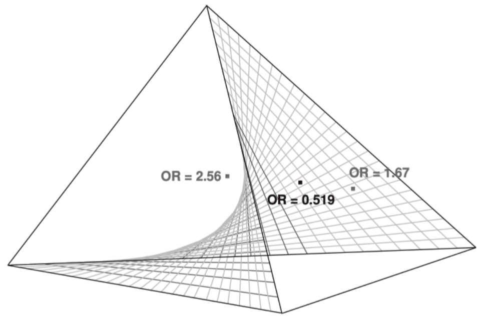


Figure 1: Statistical Manifolds, 3 non iid Probabilities.[49], Simpson's Paradox Segre Variety[50]

2.2 Connectomes

Various Neurons in our brain, carry a function forward, they interact with neurons within a group in very functional way and limit their interaction with other neurons as visual information is processed by one connectome and auditory by another. In this paper, I propose to detect hidden connectomes by group operations on AU.

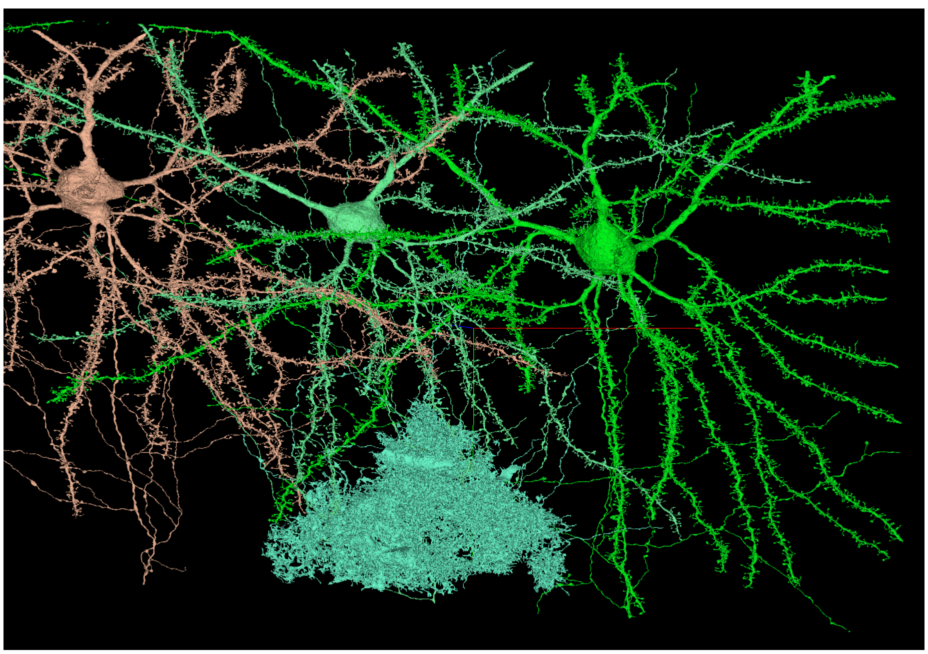


Figure 2: Functional Connectome MICrONS Dataset Consortium Mouse Visual Cortex[41]

In this method via group operations, similar sentiments, no sentiments, weak/negative sentimens, I can detect nodes that are not connected via edges but are collaboratory by behavior as a set. This set will then pass through a transformer based small graph generator that organizes graph based on operational structure over existing graph that is connected statically.

This dynamic graph will show changing alliances, impacts of various therapies and will be transient, dynamic graph. Entropy will guide probe points and just like diffusion, input/events will trigger sentiment/other outcomes that can then be used to generate these dynamic graphs, connecting these structures to long range regions of brain or organizations, which can't be accomplished by locality sensitive GNN message passing.

I will also show multiple group actions triggering different connectomes as shown below for auditory and visual connectomes in human brain. Where to probe, will be guided by lowest regions of entropy (highest probability stable core).

2.3 Metriplectic Flow: Topological Filtering for Connectome Reduction

The **Metriplectic Entropy Flow** is deployed as a **topological filter** on the graph $\mathcal{G}$. The core mechanism involves **structured graph coarsening** (via the port operator), achieving dimensionality reduction by selectively collapsing connectivity associated with high informational entropy ($\mathcal{S}$). This process yields a minimal geometric representation ($\mathcal{G}' \subset \mathcal{G}$), specifically preparing the core data for **low-cost, low-power AI**.

- **Distinction:** The objective is not traditional link prediction or standard clustering.
- **Goal:** The isolation and identification of **stable homological invariants** (functional deep structures) that define the high-definition architecture of the connectome.

2.4 Metriplectic Flow vs. Simple Statistical Filtering

2.4.1 The Failure of Static High-Entropy Filtering

A simple threshold operation (e.g., selecting the top 1% of high-entropy nodes) constitutes a static, statistical filter. It identifies nodes currently exhibiting maximum local entropy ($\mathcal{S}_{max}$) but completely disregards the system's dynamic trajectory and the conservative structure ($\mathbf{J}$) that defines stable functional connectivity. This approach risks destroying the essential topological features sought by the analysis.

2.4.2 The Metriplectic Advantage: Topological Preservation

The dynamic Metriplectic framework provides the necessary guarantees for structural preservation, as demonstrated in the following comparison:

Criterion	**Simple Static Filter (Top 1% $\mathcal{S}$)**	**Dynamic Metriplectic Flow Coarsening**
Input Data	Probability vector $\mathbf{p}$ at time t ($\mathbf{p}(t)$).	Full system dynamics $\dot{\mathbf{x}} = [J - R]\nabla H$ over Δt.
Identification	Identifies nodes currently exhibiting maximum local entropy ($\mathcal{S}_{max}$).	Identifies nodes that are **least essential** to the conservative flow.
Structural Role	Ignores the conservative structure (J); is purely statistical.	**Preserves homological invariants** defined by the J component (the "kick").
Outcome Guarantee	Does not guarantee topology preservation; functional cycles may be destroyed.	Yields a **homologically equivalent** subgraph ($\mathcal{G}'$). Minimal size for preserved function.

Refined Metriplectic Terminology and Scientific Hypothesis

The system leverages two distinct dynamic mechanisms corresponding to the conservative ($\mathbf{J}$) and dissipative ($\mathbf{R}$) components of the Metriplectic operator $\dot{\mathbf{p}} = (\mathbf{J} - \mathbf{R})\nabla H$.

1. **"Blow Up" (The J-Flow / Homological Kick)**:
   - **Action**: Identify the $\sim$ 15% highest-entropy nodes to define the *active subspace* where the structure-generating $\mathbf{J}$ flow (the Batalin-Vilkovisky kick) is applied.
   - **Purpose**: This non-dissipative component dynamically creates and maintains the functional cycles and conserved homological structures.
2. **"Blow Down" (The R-Port / Coarsening)**:
   - **Action**: Use the $\sim$ 30% threshold to determine the **Exceptional Core** that is **KEPT**, thereby rejecting 70% of the surrounding graph (the dissipative $\mathbf{R}$ component).
   - **Purpose**: This dissipative component removes nodes deemed non-essential to the preserved flow structure, achieving graph coarsening.

2.5 The Corrected Alignment Hypothesis

Given this structural clarification, the primary scientific hypothesis is:

> The nodes that I finally **KEEP** in simulation data (see simulate_entropy_p in entopy_flow_grok_2.jl), (after repeated blow downs, dissipating 70% low probability mass) (the 1% exceptional core) are, by definition, the most **stable, low-entropy,** and **essential topological features** remaining after the coarsening process.

Therefore, the expectation is that the **Lasso Reverse Map**—which identifies the most predictive features for the clinical outcome (epilepsy)—**SHOULD align strongly** with your 1% exceptional **KEPT** core.

Table 1: Expected Alignment between Dynamics and Prediction

Feature Set	Selection Basis	Expected Overlap
30% KEPT Core	Dynamic/Topological (Metriplectic **J** Component)	**HIGH**
Lasso Reverse Map	Statistical/Predictive ($\mathbf{L_1}$ Regularization)	**HIGH**

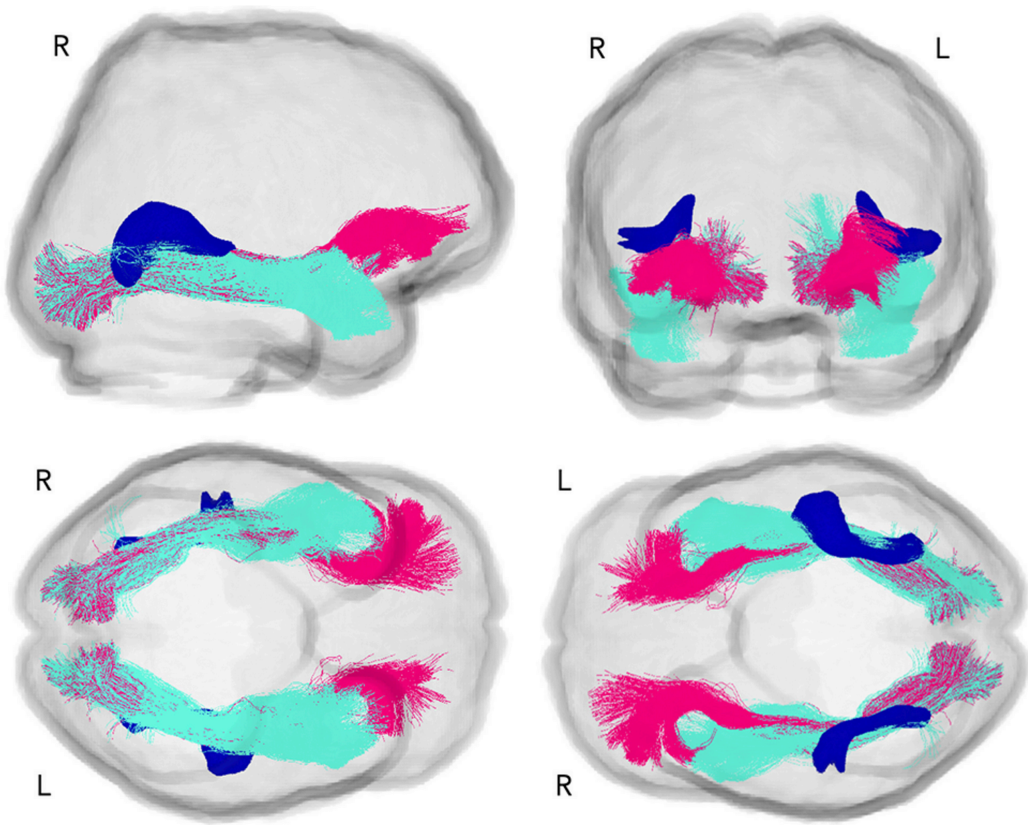


Figure 3: Collaboratory Connectomes Visual Auditory Connectomes[27]

In discrete proability space, where events are not iid, we may want to consider V-Representations which generates $\binom{2n}{n}n-1$ simplices as described in linked article H-ref Vs V-ref[48], using Minkowski-Weyl's theorem for polyhedra, one can compute H-Polyhedron (Minkowski) or V-Polyhedra (Weyl's) [54], as implemented in LattE's [29] count program using 4x4MyAlleles [51] input.

2.6 Why is removing points important?

This helps us make low power AI, low data AI, of high quality, we do not want to run gradient descent with vanishing gradients. We only want to setup regions of high **support**, which is defined as regions where cross products of $\begin{pmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \end{pmatrix}$ as positive non zero value i.e. $Supp(A_T) = (a_{ij})_{i=1..m,j=1..n}, a_{ij} \neq 0$. T indicates restriction on A. By removing points which are close to zero, we reduce lot of computation upfront[57, 32].

2.7 A Question about i.i.d. independently distributed variables – Uncertainty in AI

Most of the literature in medicine covers generation of Contingency tables by random walk, without ever getting into Information Geometry induced Algebraic Geometry [52, 53]. Researchers have characterized random walks as Fisher-Yates multivariate hypergeometric distributions where Eigenfunctions of markov chain can be generalized as multivariate Hahn polynomials [53], this information can't be directly applied to low power, low data AI as Hahn Polynomials are very compute intensive and highly numerically unstable to solve [58, 60].

Toric surface within the lattice, describes even finer details of degree of separation when we separate highly probable from less probable, and we can move this surface up and down, by adjusting our constraint in cddlib (see 4x4MyAlleles, line with 1s, in leaker [51] github repository, removing what could be interesting but not important.

This allows us to rapidly test null hypothesis with limited non-GPU-farm based compute solving NP complete problems (finding lattice point in polyhedra, based on a constraint, is NP complete) of figuring out 2x2 minor lattice paths, in polytope [55, 56] where key actors act.

Once we have Grobner bases of these set S of all possible valid constraint satisfying paths connecting 2x2 contingency matrices, we can generate all other kinds of contingency tables by linearly combining these Grobner bases, which is same as random sampling via MCMC [58, 59]. I used CoCoA system for generating these bases, which for 6x9 Contingency Table are 2x2 minors.

Table 2: Disease vs. Allele Length Contingency (Part 1: A–F)

Disease	Allele A	Allele B	Allele C	Allele D	Allele E	Allele F
A (Cystic F)	1	684	1	1	1	5
B (Sickle CD)	1	20	1	1	1	27
C (Huntington)	1	8	1	0	1	73
D (Spinal Musc. A)	1	5900	0	1	1	18000
E (Hemophilia B)	1	17	1	0	1	30
F (Thalassemia)	1	2	0	1	1	1

Table 3: Disease vs. Allele Length Contingency (Part 2: G–I)

Disease	Allele G	Allele H	Allele I
A (Cystic F)	0	0	0
B (Sickle CD)	1	0	1
C (Huntington)	1	0	0
D (Spinal Musc. A)	0	0	0
E (Hemophilia B)	0	0	1
F (Thalassemia)	0	1	1

```
> data_alleles_AF <- matrix(c(
+   1, 684, 1, 1, 1, 5,
+   1, 20, 1, 1, 1, 27,
+   1, 8, 1, 0, 1, 73,
+   1, 5900, 0, 1, 1, 18000,
+   1, 17, 1, 0, 1, 30,
+   1, 2, 0, 1, 1, 1
+ ), nrow = 6, byrow = TRUE, dimnames = list(
+   Disease = c("A", "B", "C", "D", "E", "F"),
+   Allele = c("A", "B", "C", "D", "E", "F")
+ ))
> # Example using your 3x9 data (or your full 6x9 data)
> fisher.test(data_alleles_AF, simulate.p.value = TRUE, B = 10000)

   Fisher's Exact Test for Count Data with simulated p-value (based on 10000 replicates)

data:  data_alleles_AF
p-value = 9.999e-05
alternative hypothesis: two.sided

> print(data_alleles_AF)
       Allele
Disease A    B C D E     F
      A 1  684 1 1 1     5
      B 1   20 1 1 1    27
      C 1    8 1 0 1    73
      D 1 5900 0 1 1 18000
      E 1   17 1 0 1    30
      F 1    2 0 1 1     1
> |
```

Figure 4: Null Hypothesis Testing indicates Strong correlation, possible mRNA transcription error

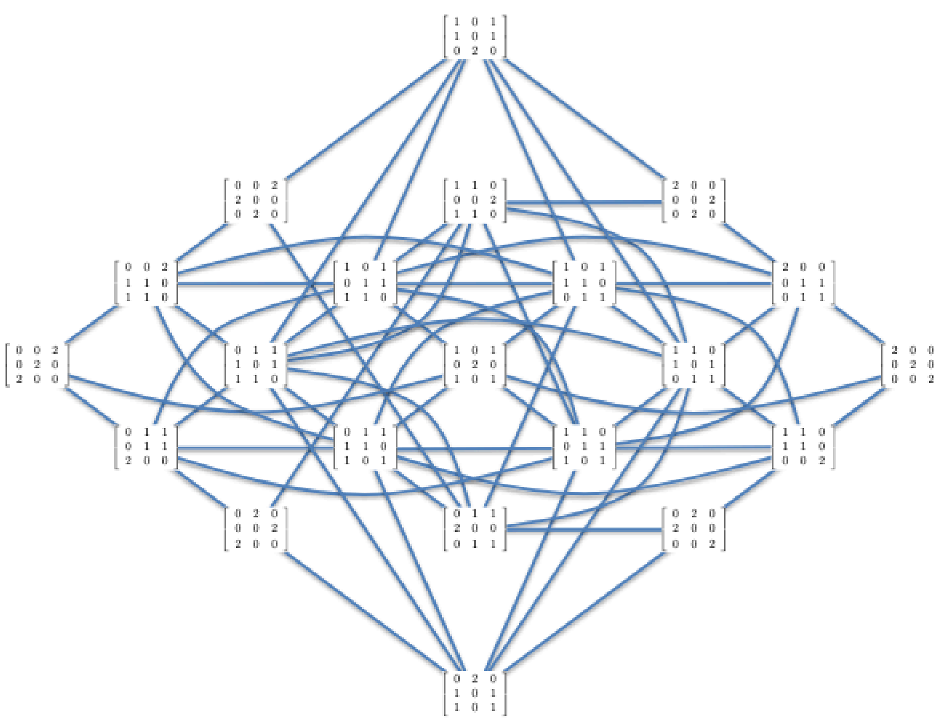


Figure 5: Random Walk on Contingency Table of size 3x3 with 2x2 minors flipping signs in row/column, keeping marginal sum, same [56]

Disease \ Allele	A	B	C	D	E	F
A	1	684	1	1	1	5
B	1	20	1	1	1	27
C	1	8	1	0	1	73
D	1	5900	0	1	1	18000
E	1	17	1	0	1	30
F	1	2	0	1	1	1

Figure 6: This Contingency Table has numerous paths due to very high entry count for Row D Spinal Muscular Atrophy, We will cull lattice points of low probabilities by using constraints programming and by moving toric (2x2 minors) surface up.

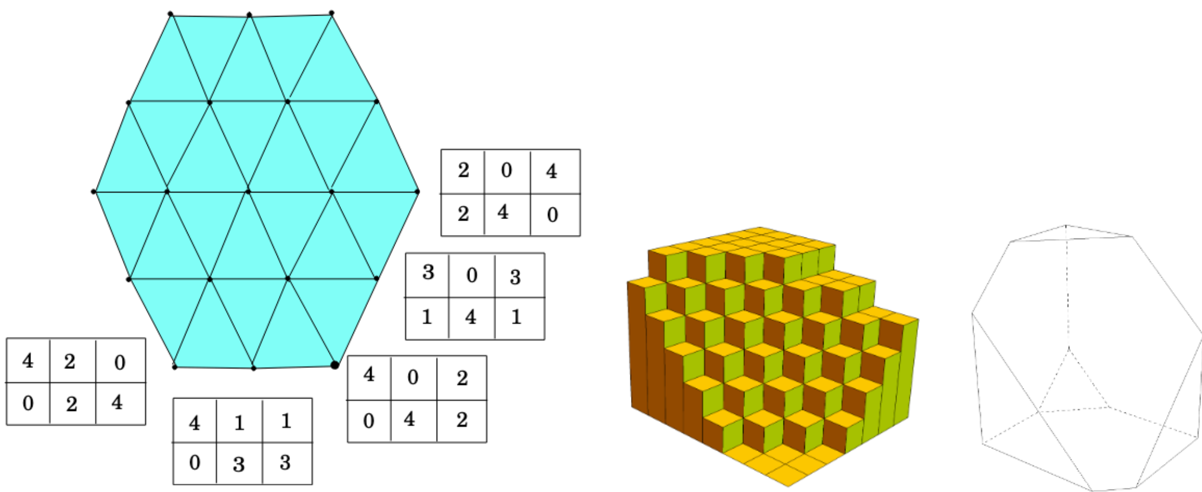


Figure 7: 2x3 minors with positive det become lattice points, and edges describe existence of $+ve$ transition probabilities, eigenfunctions of MCMC become Hahn Polynomials[56, 55, 53], by truncations one can remove large spaces of dense data of low probability [57], higher values of column conditionals $(p_{11}p_{12}/p_{12}p_{21})$ show higher chance of transition to next column, higher values of row conditionals $(p_{11}p_{21}/p_{12}p_{22})$ indicates higher chance of row transition [50], *this helps us generate high quality data for hypothesis testing, while performing low compute AI*
.

3 Statistical Inference using commutative algebra

The R package algstat calls 4ti2 and LattE to perform exact statistical inference for log-linear models.

Since the underlying calculations for exact inference involve complex commutative algebra and discrete geometry, algstat delegates the "heavy lifting" to these tools while managing the data and results within the R environment.

3.1 How `algstat` Uses `4ti2` and `LattE` for Log-Linear Analysis

Key to computability is Latte's contraint solver in count program, deciding how big is the search space of generated tables (21956000). We remove low quality data from here and only keep high quality transitions.

The number of possible contingency tables satisfying the marginal constraints for the independence model is calculated using the `algstat` package, which calls the external `LattE` program:

```
number_of_tables <- algstat::count_tables( table = data_alleles_AF3 )
```

Above code indicated that 21956000 tables were generated. File 4x4MyAlleles in Github repository Leaker,line 8, controls how linear program culls low probability dense clusters of lattice points.

```
-601 1 1 1 1 1 1 1 1 1
```

3.2 1. `4ti2` for Generating the Markov Basis (Movement)

`algstat` uses the `4ti2` programs—specifically `loglinear` and `markov`—to define the algebraic constraints of the model and generate moves necessary for sampling.

Model Definition

- `algstat` reads simplified R matrix and the log-linear formula (e.g., $\sim$ `Row` + `Col`).
- It automatically creates the necessary `.in file` for `4ti2` that defines the dimensions of your table and the marginals you want to hold fixed (e.g., row and column totals for independence).

Basis Generation

- `algstat` executes the `4ti2` command `loglinear` via R's `system()` call.
- `loglinear` computes the **Markov basis** (stored in a `.mat file`), which is the minimal set of integer vectors (moves) needed to transform any table that satisfies the fixed marginals into any other table that satisfies the same marginals.

Sampling (MCMC)

- If you call `algstat::mcmc_independence()`, `algstat` executes the `4ti2` command `markov`.
- `markov` uses the generated basis to walk a Markov Chain, sampling tables from the exact conditional distribution defined by the fixed marginals.

3.3 2. `LattE` for Counting the Sample Space (Enumeration)

`algstat` uses the `LattE` program, specifically `count`, when you need to know the **exact number** of possible tables that satisfy your model's marginal constraints (the size of the sample space). **It is this count we cull, by changing linear constraint, forcing toric-surface (Fig - 1) up and down removing lattice points of low probability transitions, which could be in millions.**

LattE Setup

The method involves finding the volume of a convex polytope defined by the marginal constraints, which corresponds to the number of non-negative integer solutions (tables).

Execution and Result

- `algstat` executes the `LattE` command `count` via R's `system()` call.
- The `count` program returns the total number of distinct contingency tables with your specific fixed marginals. This is typically only feasible for very small tables or low marginal totals due to the computational complexity.

```
> print(data_alleles_AF3)
       Allele
Disease A   B C D E  F
      A 1 684 1 1 1  5
      B 1  20 1 1 1 27
      C 1   8 1 0 1 73
> loglinear(model = ~Disease + Allele, data = data_alleles_AF3)
Care ought be taken with tables with sampling zeros to ensure the MLE exists.
Computing Markov moves (4ti2)... done.
Running chain (C++)... done.
Call:
loglinear(model = ~Disease + Allele, data = data_alleles_AF3)

Fitting method:
Iterative proportional fitting (with stats::loglin)

MCMC details:
N = 10000 samples (after thinning), burn in = 1000, thinning = 10

                    stat p.value se mid.p.value
      P(table)                0  0           0
   Pearson X^2 616.3944       0  0           0
Likelihood G^2 466.396        0  0           0
 Freeman-Tukey 516.0105       0  0           0
  Cressie-Read 525.4396       0  0           0
    Neyman X^2 2009.7285      0  0           0
> 
```

Figure 8: 10000 Contingency Tables were generated with 1000 burn in, **showing strong correlation** among events [35]

```
# use P ::= QQ[x,y,z];
# G := GBasis(Ideal(x11 -y1*z1, x12 - y1*z2, x13 - y1*z3, x21 - y2*z1, x22 - y2*z2, x23 -y2*z3, x31 - y3*z1, x32 - y3 * z3, x33 - y3 * z3));
--> ERROR: Cannot find a variable named "x11" in scope
--> G := GBasis(Ideal(x11 -y1*z1, x12 - y1*z2, x13 - y1 ...
-->                   ^^^
# use P := QQ[y1,y2,y3,z1,z2,z3,x11,x12,x13,x21,x22,x23,x31,x32,x33];;
--> WARNING: Write ::= instead of := to avoid this warning
--> use P := QQ[y1,y2,y3,z1,z2,z3,x11,x12, ...
-->       ^^
# use P ::= QQ[y1,y2,y3,z1,z2,z3,x11,x12,x13,x21,x22,x23,x31,x32,x33];;
# G := GBasis(Ideal(x11 -y1*z1, x12 - y1*z2, x13 - y1*z3, x21 - y2*z1, x22 - y2*z2, x23 -y2*z3, x31 - y3*z1, x32 - y3 * z3, x33 - y3 * z3));
# G;
[y3*z3 -x33,  x32 -x33,  y2*z3 -x23,  y1*z3 -x13,  y2*z2 -x22,  y1*z2 -x12,  y3*z1 -x31,  y2*z1 -x21,  y1*z1 -x11,  z3*x22 -z2*x23,  z3*x12 -z2*x13,  z3*
x31 -z1*x33,  z3*x21 -z1*x23,  z3*x11 -z1*x13,  y3*x23 -y2*x33,  y3*x13 -y1*x33,  y2*x13 -y1*x23,  z2*x21 -z1*x22,  z2*x11 -z1*x12,  y2*x12 -y1*x22,  y3*
x21 -y2*x31,  y3*x11 -y1*x31,  y2*x11 -y1*x21,  x23*x31 -x21*x33,  x13*x31 -x11*x33,  x13*x22 -x12*x23,  x13*x21 -x11*x23,  x12*x21 -x11*x22]
# use P := QQ[y1,y2,y3,y4, z1,z2,z3,z4, x11,x12,x13,x14, x21,x22,x23,, x24, x31,x32,x33, x34];;
--> WARNING: Write ::= instead of := to avoid this warning
--> use P := QQ[y1,y2,y3,y4, z1,z2,z3,z4,  ...
-->       ^^
--> ERROR: I was expecting an identifier but I've found the operator/symbol ","
--> ... ,z2,z3,z4, x11,x12,x13,x14, x21,x22,x23,, x24, x31,x32,x33, x34];;
-->                                            ^
# use P := QQ[y1,y2,y3,y4, z1,z2,z3,z4, x11,x12,x13,x14, x21,x22,x23, x24, x31,x32,x33, x34];;
--> WARNING: Write ::= instead of := to avoid this warning
--> use P := QQ[y1,y2,y3,y4, z1,z2,z3,z4,  ...
-->       ^^
# use P ::= QQ[y1,y2,y3,y4, z1,z2,z3,z4, x11,x12,x13,x14, x21,x22,x23, x24, x31,x32,x33, x34];;
# G := GBasis(Ideal(x11 - y1z1, x12 - y1*z2, x13 - y1*z3, x14 - y1*z4, x21 - y2*z1, x22 - y2*z2, x23 -y2*z3, x24 - y2*z4, x31 - y3*z1, x32 - y3 * z3, x33
 - y3 * z3, x34 - y3*z4));
--> ERROR: Cannot find a variable named "y1z1" in scope
--> G := GBasis(Ideal(x11 - y1z1, x12 - y1*z2, x13 - y1*z3, x1 ...
-->                         ^^^^
# G := GBasis(Ideal(x11 - y1*z1, x12 - y1*z2, x13 - y1*z3, x14 - y1*z4, x21 - y2*z1, x22 - y2*z2, x23 -y2*z3, x24 - y2*z4, x31 - y3*z1, x32 - y3 * z3, x3
3 - y3 * z3, x34 - y3*z4));
# G;
[y3*z4 -x34,  y2*z4 -x24,  y1*z4 -x14,  y3*z3 -x33,  x32 -x33,  y2*z3 -x23,  y1*z3 -x13,  y2*z2 -x22,  y1*z2 -x12,  y3*z1 -x31,  y2*z1 -x21,  y1*z1 -x11,
  z4*x33 -z3*x34,  z4*x23 -z3*x24,  z4*x13 -z3*x14,  z4*x22 -z2*x24,  z4*x12 -z2*x14,  z4*x31 -z1*x34,  z4*x21 -z1*x24,  z4*x11 -z1*x14,  y3*x24 -y2*x34,
  y3*x14 -y1*x34,  y2*x14 -y1*x24,  z3*x22 -z2*x23,  z3*x12 -z2*x13,  z3*x31 -z1*x33,  z3*x21 -z1*x23,  z3*x11 -z1*x13,  y3*x23 -y2*x33,  y3*x13 -y1*x33,
  y2*x13 -y1*x23,  z2*x21 -z1*x22,  z2*x11 -z1*x12,  y2*x12 -y1*x22,  y3*x21 -y2*x31,  y3*x11 -y1*x31,  y2*x11 -y1*x21,  x24*x33 -x23*x34,  x14*x33 -x13*
x34,  x24*x31 -x21*x34,  x14*x31 -x11*x34,  x23*x31 -x21*x33,  x13*x31 -x11*x33,  x14*x23 -x13*x24,  x14*x22 -x12*x24,  x13*x22 -x12*x23,  x14*x21 -x11*x
24,  x13*x21 -x11*x23,  x12*x21 -x11*x22]
```

Figure 9: CoCoA computes grobner bases for polynomials describing cross products as x11 - y1z1 (p11p22) etc... producing 2x2 minors as Grobner bases once y and z terms are removed.[35]

Adding data that is too sparse, generating even 1336859882750 tables to explore, still shows strong co-relation and case for rejecting null hypothesis (diseases are independent of underlying mRNA related errors, driven as root cause in all).

```
      Allele
Disease A    B C D E    F
      A 1  684 1 1 1    5
      B 1   20 1 1 1   27
      C 1    8 1 0 1   73
      D 1 5900 0 1 1 1800
> loglinear(model = ~Disease + Allele, data = data_alleles_AF4)
Care ought be taken with tables with sampling zeros to ensure the MLE exists.
Computing Markov moves (4ti2)... done.
Running chain (C++)... done.
Call:
loglinear(model = ~Disease + Allele, data = data_alleles_AF4)

Fitting method:
Iterative proportional fitting (with stats::loglin)

MCMC details:
N = 10000 samples (after thinning), burn in = 1000, thinning = 10

                    stat p.value se mid.p.value
      P(table)                 0  0           0
   Pearson X^2 708.6825        0  0           0
Likelihood G^2 558.3817        0  0           0
 Freeman-Tukey 705.9204        0  0           0
  Cressie-Read 570.8007        0  0           0
    Neyman X^2 5022.1439       0  0           0
> number_of_tables_4 <- algstat::count_tables(
+   table = data_alleles_AF4
+ )
> number_of_tables_4
[1] "1336859882750"
```

Figure 10: Presence of D (Spinal Musc. Atrophy) with much larger number of elements still shows independence model fits poorly even though there is large difference in Chi square (Pearson), which is sensitive to discrepancy and Likelihood data, which penalizes discrepancy less severely, ***indicating strong association***.

4 Dataset, Low Data Low Power AI

Current AI development is dominated by the unsupervised training of large language models on massive, high-quality datasets. Today's state-of-the-art LLMs have already been trained on virtually all accessible data. This process, which requires immense computational resources—involving hundreds of thousands of GPUs running for over a year—produces a transformer model whose knowledge is strictly confined to its training data. Consequently, these models are prone to hallucination. They lack a mechanism to distinguish fact from fiction, often generating plausible-sounding but inaccurate text by statistically predicting the next most likely token [45].

The batch training paradigm lacks scalability for detecting obfuscated gene-therapy interaction patterns over extended periods. This is exacerbated by data scarcity, a common consequence of institutional non-disclosure. Since only final outcomes are available, I am using the following contingency table to assess whether ongoing events can form a valid hypothesis. A similar contingency table based tabular structure is commonly used in pharmacology to analyze gene interactions adopted by Agresti [5, 20].

4.1 Example

From our Contingency matrix, we extract 2x3 contingency matrix for Cystic Fibrosis and Sickle Cell D, both had Allele B (684, 20) and Allele F(5,27), both have resources for which a regime change may be under consideration. In flattened row major form $\begin{pmatrix} 1 & 684 & 5 \\ 1 & 20 & 27 \end{pmatrix}$ $T = (1, 684, 5, 1, 20, 27)$. If we were to restrict moves to strictly positive (i.e. only non zero moves), our table size only reduces by 2 as numbers are large and only resources are carrying 1/0. We then compute high probability corridors by putting edge weights $(x_{ii}x_{i'j'} - x_{ij'}xi'j = w_{ii'jj'})$ for $i \mapsto i', j \mapsto j'$ row column transition. This probability model on transition assigns each table T a probability $\pi(T) \propto \prod_{ij} \theta_{ij}^{t_{ij}}$, removing paths of lower value, cull subregions carrying low probaility mass, while retaining connectivity/MCMC Sampling.

Nodes score is probability mass $\pi(T)$. Edge score is f(T, T') = $min\{\pi(T)P(T \propto T'), \pi(T')P(T' \propto T)\}$ We solve for max path score as $\sum logP$ We can cull lattice points by supplying linear programming constraint in 4x4MyAlleles file, however to find paths, at low compute and low power, it is best to apply common sense, pick smaller 2,3 tables and run chat_Gpt_generated_path_generation.py program, based on my prompts for directing strictly increasing moves in S set, disallowing zeroes, which generated following path. One can easily write this program as well, using Djikstra's algorithm for finding highest log P generating path.

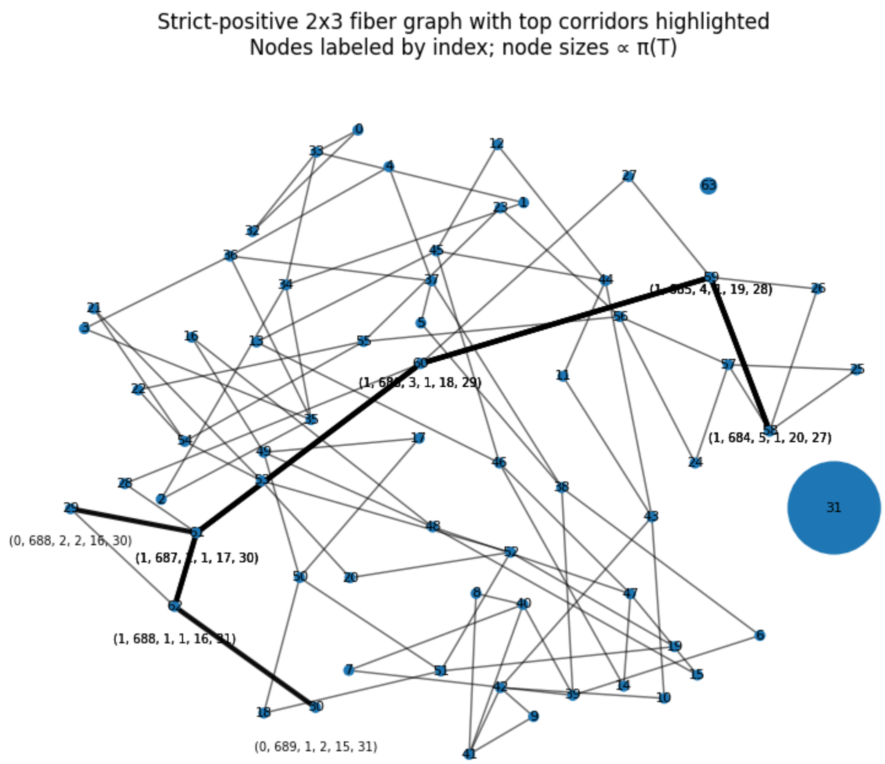


Figure 11: Important Connections in Contingency Table – ChatGPT generated Djikstra's max log P path/-graph, code; Other variations in 2x2 minors (62 of them) of 2x3 Contingency table and related connections are not as important as Allele B/Allele F connection, due to strict positiveness, we may miss-out some even higher probability paths going via 0.

Also normalizing data is very important, here node 31 has largest circle, because 684, Allele B generated highest probability based on seed count.

Using $x_{ii}x_{i'j'} - x_{ij'}xi'j$ as toric ideal of sublattice $I_{2,3}$ for fiber $\mathcal{F}$ as $\mathcal{C}(2,3)$:

$$\begin{pmatrix} t_{(ij)} & t_{(i+1,j)} \\ t_{(i,j+1)} & t_{(i+1,j+1)} \end{pmatrix} \mapsto \begin{pmatrix} t_{(ij)}+1 & t_{(i+1,j)}-1 \\ t_{(i,j+1)}-1 & t_{(i+1,j+1)}+1 \end{pmatrix}$$

with mapping $(t) \mapsto ((1-t).T_{ij}^{(k)} + t.T_{i'j'}^{'(k+1)} \in \mathbb{R}^{2x3})$. With in toric co-ordinates becomes:

$$x_{ij}(t) = x_{ij}^{(k)}.\Big(\frac{x_{ij}^{(k+1)}}{x_{ij}^{(k)}}\Big)^t$$

Analogous to culling lattice points in a polyhedron, we have eliminated low-probability regions while retaining the main mass and its internal connectivity. By interpreting $\pi(.)$ as flow, we can then identify a min-cut that separates the high-mass core from the low-mass periphery.

We now project 6D (there are 6 elements in 2x3 minor) to 3D for visualization, see toric.py in github Leaker repository Leaker

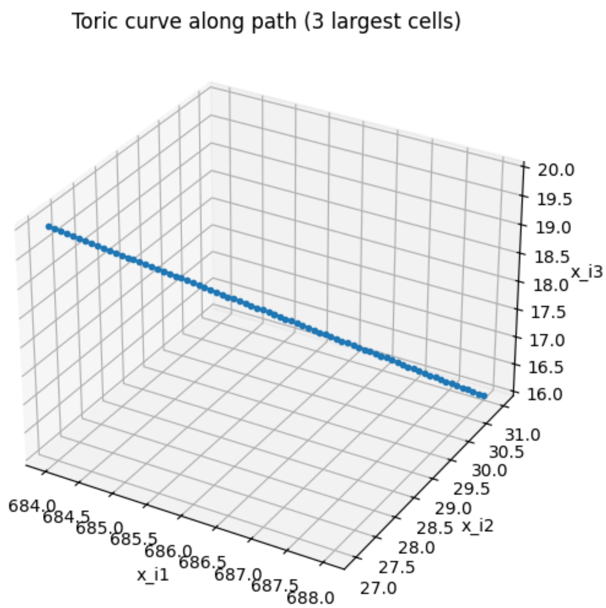


Figure 12: By coordinate transformation on 2x2 minors (determinantal variety), we get this toric curve 6D projected to 3D, for bold path [(1, 684, 5, 1, 20, 27), (1, 685, 4, 1, 19, 28), (1, 686, 3, 1, 18, 29), (1, 687, 2, 1, 17, 30), (1, 688, 1, 1, 16, 31)] in ***Fig 11***.

Key intuition is*, paths can't have cycles as binomial edge ideals will then form syzygies (i.e. linear combination of generators of Ideal will cancel out*, **syzygy** will indicate that some paths are reducible due to presence of cycles [64][61][62][63]. This becomes very helpful in eliminating large symmetries *equivariant Grobner Basis*[64].

When we map these events over to observations on GNN on ogbl-vessel dataset, there too, we will need to cull node symmetries by node labeling tricks[17][13]/Forman Ricci Curvature based graph rewiring performing Stochastic Discrete Ricci Flow (SDRF), breaking local symmetries[72]*.*

5 Metriplectic Flow vs. Local GNN Aggregation

The Metriplectic approach explicitly moves beyond the local feature aggregation mechanisms—the core function of standard Graph Neural Networks (GNNs)—which primarily aim to create similar embeddings for locally similar nodes or predict local links.

Instead, my approach utilizes global thermodynamic dynamics (the $\mathbf{J} - \mathbf{R}$ flow) to:

1. **Isolate and Define the Essential Structure:** The method focuses on identifying the $\sim 30\%$ **stable, low-entropy core** ($\mathcal{G}'$).
2. **Analyze Global Function:** The goal is to isolate the **large-scale, global topological structures** that are responsible for sustaining a complex functional outcome (e.g., epilepsy).

5.0.1 Comparison of Focus

Table 4: Standard GNNs vs. Metriplectic Flow

Aspect	**Standard GNNs**	**Metriplectic Flow**
Primary Goal	Local feature discrimination; link prediction	**Global structure detection**; stable core isolation
Core Mechanism	Local message passing (1–3 hops)	Global $(\mathbf{J} - \mathbf{R})$ dynamics on probability manifold
Operating Scale	Node neighborhoods	**Graph-wide cycles**; long-range coherence
Output	High-dimensional embeddings	Low-entropy core subgraph $\mathcal{G}'$ (**KEPT**)

For our 2x3 contingency table, Grobner basis are 2x2 minors, one can eliminate paths, by placing structural zeros [0] in contingency tables [55]. These symmetries often arise when multiple actors, genes act similarly in varying conditions understudy. For our ogbl-vessel dataset, symmetries exists in right vs left brain, and for our Contingency Table, we see actors trained, follow similar paths (Allele B/Allele F, in a highly correlated fashion).

5.1 Existing Structures

To model existing visible structures of various players, acting via obfuscated links at scale an obg vessel dataset is used as it provides hierarchical graph data mapping entire region of mouse brain [10, 13]. Approaches utilizing GNNs go on to predict link using labeling tricks on subgraphs of nodes under consideration [19, 17]. I add sentiments to nodes to generate hypergraph invisible structures characterized by algebraic sets. These sets then engage in highlighting visible parts of predator prey ODEs, formed by observable observations in contingency table. I do not assume that ogb-vessel dataset has all known actors. We do have information about single role played by nodes, secondary role is only visible via their interactions (genes triggering interactions via spike proteins or social media posts to manufacture sentiments). This approach of rattle based detection of connections is very similar to fMRI helping to find connectomes by detecting synchronized, or correlated, activity between different brain regions through the BOLD signal.

First let us look at dataset and then see what SEAL[47] predicts by labeling tricks and why it is inadequate for our connectome detection. ogbl-vessel database comes in converted compressed numpy/torch tensors (npz, pt files). One can load and run seal by splitting data into training validation and test but as is, viewing this data requires having access to voreen volume rendering tools (voreen[12]), these tools have their own pipelines to render these graphs and may require QT5 compilation using older boost libs which are kept inside qt5 qtconnection code tree. On Mac OS, QT5 can be compiled after adjusting boost, however voreen or voreen_tool will not be buildable as they require cpufetch and auxv.h (Auxilary vector to kernel to pass data – not supported by MacOS) file support. Do not fall for voreen build for MacOS instructions saying *"Since our development is cross plattform a compilation from source is possible, but you may encounter some difficulties"*.

First get data which was used to get compressed numpy and torch tensors from VesselGraph[14]. This data is organized in BALBc (5.35 million edges, 3.54 million nodes, inbred white mouse, good for immune response testing), C57BL_6 (inbred black mouse good for cellular immune response testing, genetically modified – used for obesity, diabetes, neurobiology and genetics related experiments), CD1-E (outbred swiss mouse, good for testing toxicology, safety, and efficacy testing – has larger size, low anxiety, has intracranial collateral vascularization, enabling brain to continue to function if some edges are occluded – good for studying secondary/traumatic brain injury pathways, has long connections and unregulated Cyclin D1 protein that trigers inflammation, and neuro degeneration) [15].

I picked Raw BALBc_no1, node and edges .csv files, produced by voreen. I then ran can be fed to VTK script to generate a vtp file for Paraview rendering [16]. See build_vtp.py file for details in Leaker Github repository.

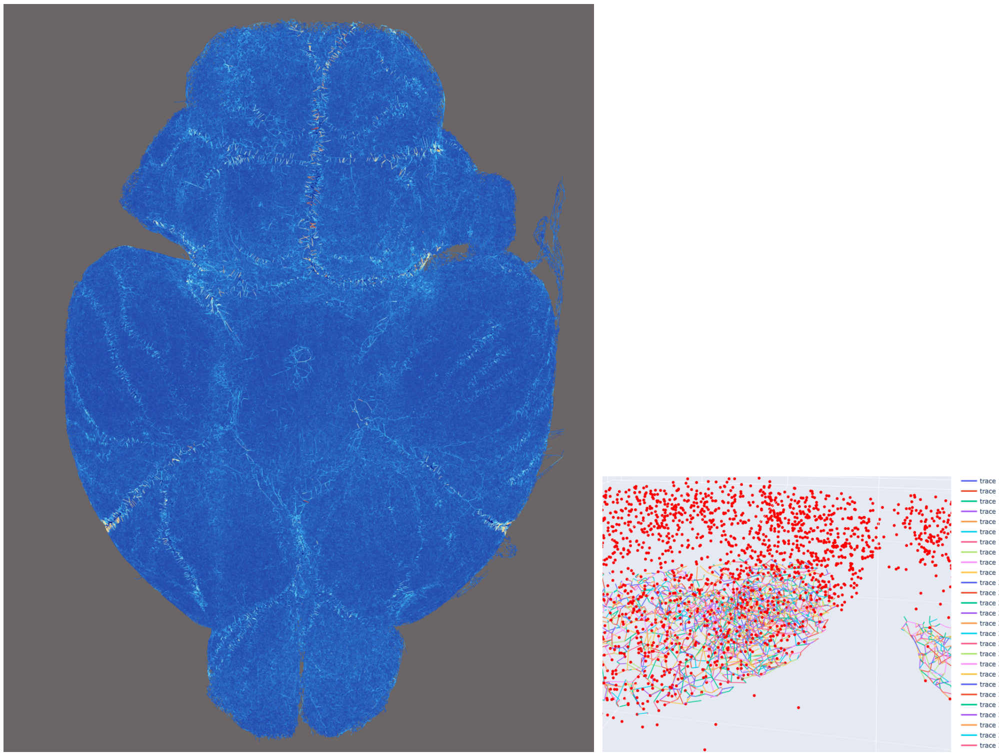


Figure 13: Left in blue - BALBc_no1 Paraview View of ogbl-vessel dataset, 5.35 million Edges and 3.54 million nodes on Laptop taking 1.8 GB memory, Right Python based Plotly ran all night, taking 18+ GB memory and was not able to render. See gen_view.py for reduced representation script of 20,000 edges of BALBc_no1, picture on right

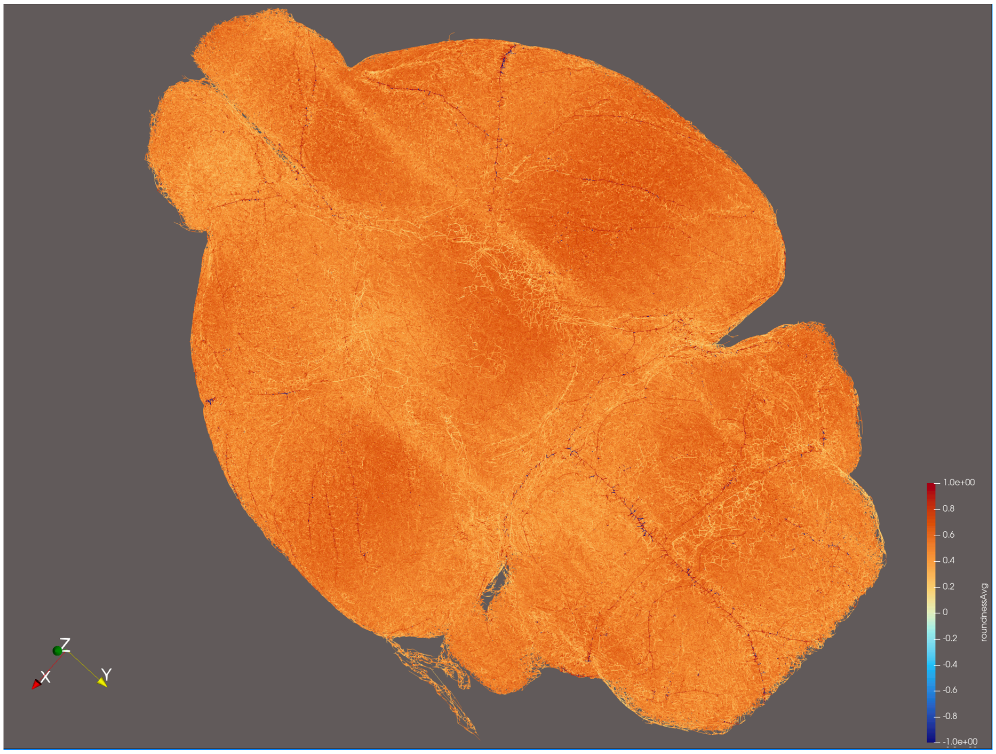


Figure 14: BALBc Mouse Brain Brain, colored by roundedness of edges, Darker/rounder lines are showing Circle of Willis (network of arteries).[11]

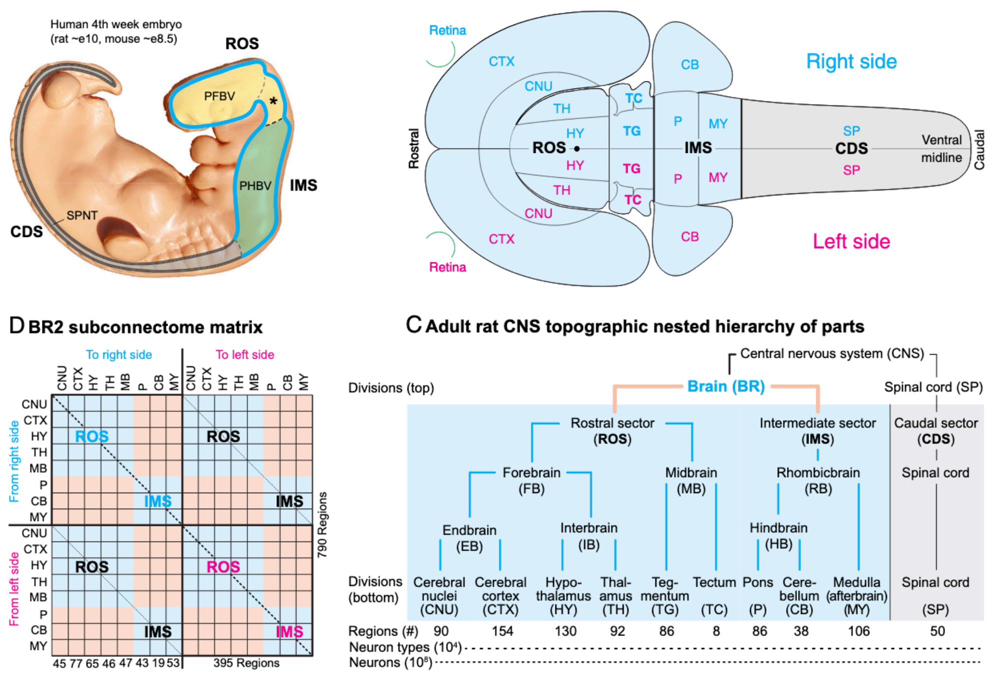


Figure 15: Observable Node Structures acting via well defined roles and interfaces, we overlay secondary observations via node labeling tricks, GNN using SEAL can predict "links" (not structures) in this dataset [17, 46, 47, 18].

5.2 SEAL - 2 hop Link Prediction vs long functional connectomes representing deep structures

As reported in the Vessel Graph paper [13], the SEAL method [47] achieved roughly 90 percent link-prediction accuracy, significantly outperforming GCN and GraphSAGE (which was at around 60 percent accuracy). This improvement stems from SEAL's Double radius Node Labeling (DRNL) techniques and its ability to capture graph symmetries, typically converging within a single epoch. As the authors note, "SEAL almost converges in the first epochs while inducing significant inductive bias, making it the superior model for our spatial and hierarchical graph."[13].

All AI learning is Contextual and local.

Authors of SEAL [47] acknowledge "Our results build the foundation for learning heuristics from local subgraphs, as they imply that local enclosing subgraphs already contain enough information to learn good graph structure features for link prediction which is much desired considering **learning from the entire network is often infeasible**. " We need to find a way to find links spanning across large distances as connectomes work that way.

Deep structures interact globally, ***weaving connectomes*** that extend across multiple regions and link, sets that initially seem unconnected. *We can't rely on 2 hops DRNL/Distance Encoding (DE) to predict links forming connectomes.*

5.3 Arithmetic Units on Regions/Nodes/Voxels

I am using Julia's Finset to apply functions on various set to simulate wave of operations (sentiments farming). I use Allen Brain Atlas to label regions [73], [74], [75].

Observer trying to find connectomes, has no idea about underlying architecture of actors and their methods. I train a model, where I build observations attribution to various regions (for mRNA it could be gene present and their impact on various regions). Observer only sees outcomes and then in order to act observer needs to locate connectomes. These connectomes are built dynamically and have little dependence on Graph's physical structure. These links in connectomes are formed via outside interactions, which are un-knowable for an observer watching events.

To analyze, we assign regions for reference using labels at Kaggle for BALBc_no1 at BALBc_no_1_Allen_Atlas_Regions. Using vtk (PyVista has strong assumptions about disconnected isolated edges, so it fails) in script build_coded_filtered.py, I have hierarchical regions interacting via unknown deep structures.

We will not know these regions in advance, we will also not know their interacting pathways as we can't apply Diffusion Tensor Imaging (DTI) to collect what happens when certain actions are performed, we will build these "***connectomes***" via AU's algebraic operation. This can be considered as regions/organizations who get active when blood oxygen level goes up (+ sentiments, -ve sentiments, no reactions – i.e change in entropy) signals, emanating from various x,y,z locations, in brain. I will only use region key as a set identifier, connectomes will cross many of these regions to drive given functional objective creating dynamic graphs describing flow of entropy.

These regions may include neurons—whether influenced by various actors or by factors such as mRNA-related protein expression that crosses the blood–brain barrier—that participate in multiple processes depending on the governing functional pathway, a kind of "functional path singularity." Because we lack Diffusion Tensor Imaging (DTI) data to map these pathways directly, I'll rely on entropy and "lift" to identify and disentangle the overlapping routes, using only observed outcomes and preceding events.

We can now do AU based processing on these regions to build a probabilistic inference model.

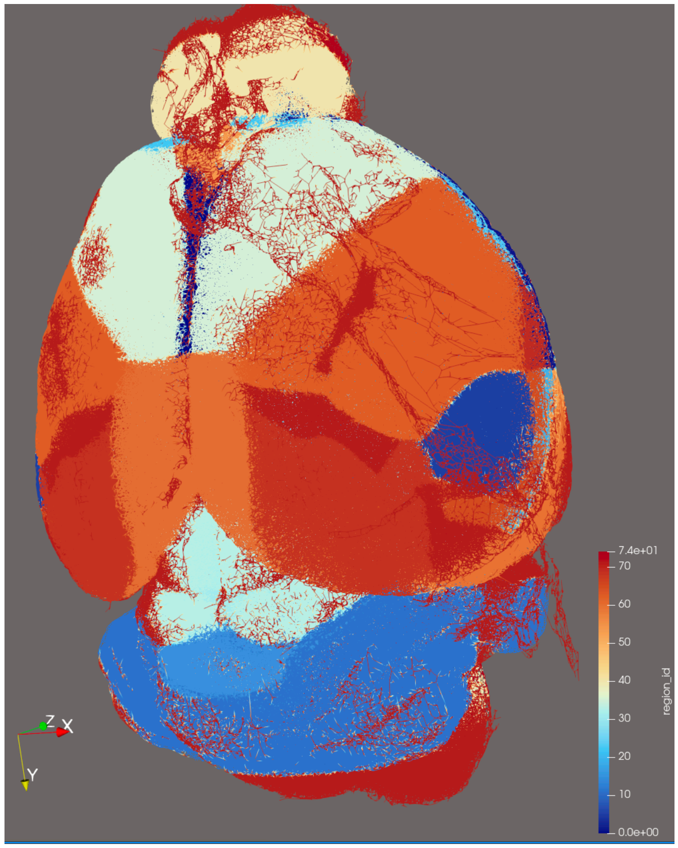


Figure 16: Hidden Structures that only emerge based on Structural Coherence.

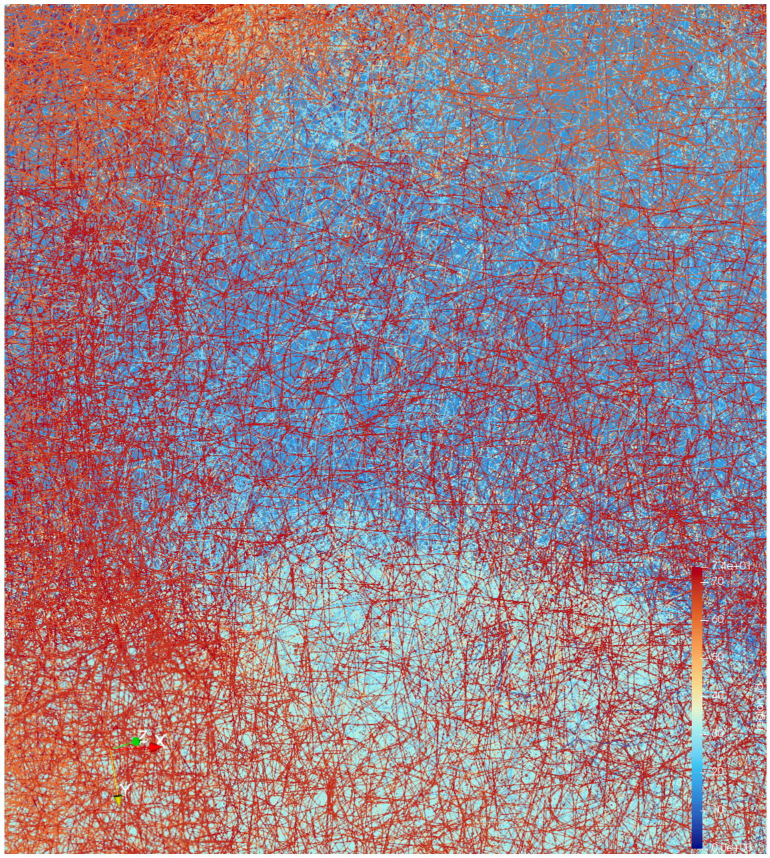


Figure 17: Hierarchical Graph with Structural and Functional design. One can only see structure in this picture, connectomes will reveal functional structure

5.4 Building Deep Hidden Structure - 2 sided approach

Start with Null Hypothesis fron contingency table and gather observable interactions on structured node data which models extra interactions of nodes, motivated by obfuscated goals. Say I observe, social media posts to destabilize a country, sow confusion and chaos. Based on scale, and other observable events, and hypothesis indicating many observable events, I establish strong correlation among actors in contingency table forming a polytope lattice, which can also be approximated by Hahn polynomials, of 2x2 minors as nodes, connected by paths of random walks on contingency matrix, generating MCMC samples and links indicating positive probability flow.. Using Groebner bases, these million of paths collapse to 2x2 minors provided there are no structural zeros (regions of improbable interactions)[55]. These 2x2 minors can then be used to generate all paths using linear combinations as they form a ring.

5.4.1 Generate Data - 2x2 Minor Lattice Points By Random Walk

By deploying randomly adding subracting 1 from row column data as a set operation $\begin{pmatrix} + & - \\ - & + \end{pmatrix}$ one can generate new contingency table to evaluate how intertwined our events really are.

5.5 Connecting Structures using RL

Build from observations connecting obfuscated links/structures (data from contingency table) to observable structures (ogb-vessel data)

6 Arithmetic Units

Arithmetic Unit or universe describes sets where functions can act upon. I use Catlab.jl and Algebraic-Dynamics.jl and AlgebraicRelations.jl Julia packages to model these predator prey actions (see Allele B vs Allele F chart) [43, 44]. Basic ideas is related to modeling "rattling". Whatever is working in harmony across network is "connected". This connection is identified by applying push-out stimulus functor on Finsets (only on reduced 1 percent sets this could be possible). This same methodology can be used to connect mRNA gene alleles to observable symptoms/via immune response.

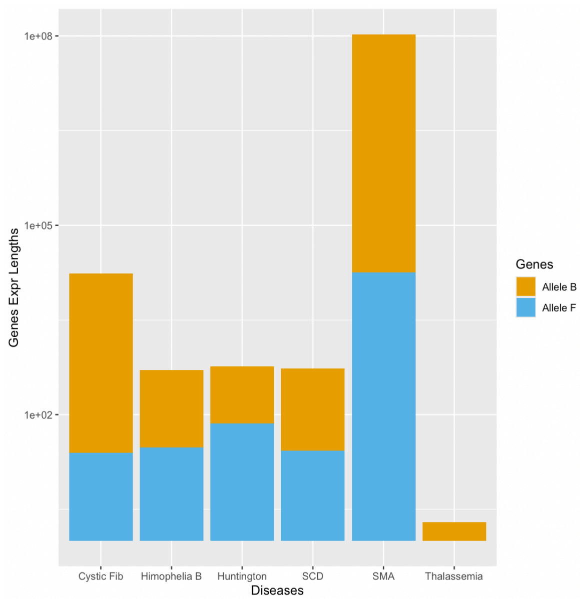


Figure 18: AU : Gene Expression Lengths modeled using normal sets/Algebraic Julia Finsets are too expensive for millions of data nodes/edges

7 Arithmetic Units and Entropy Flow using Grestenhaber and Batalin-vilkovisky structures

We are not looking at Physical nearby interactions, but long range information flows, modeled via entropy. In almost all structures which are engaged in functional tasks, such as Governments organizing attacks or vaccines triggering immune response, we usually see entropy flows in networks. Sudden spike in activity, such as blood oxygen level dependent functions detected via fMRI (BOLD), or flood of tweets, in-organic demonstrations to sway public opinions, designed to facilitate action. This information flows through networks creating trace of nodes engaged in activating response or facilitating action readiness.

7.1 Gerstenhaber algebra

Imagine a graph G = (v, e), where edge e carry flows such as probability mass, information. These flows can have various directions, often meeting at nodes where nodes will contribute actions in an unknown ***singular*** way.

Singularity at a node here is describing multiple incoming flows without a clear outgoing path. Cycles or loops can also make the flow undefined. There could also be points in data were conservation of entropy/probability laws may fail.

We want ***algebraic structure*** that can help "resolve" or systematically handle these singularities [76]

Gerstenhaber algebra is a graded vector space.

$$A = \bigoplus_{n} A^n$$

equipped with: A graded commutative product $A^i \otimes A^j \to A^{i+j}$
A graded Lie bracket

$$[-,-] : A^i \otimes A^j \to A^{i+j-1},$$

satisfying:

- **Graded antisymmetry**
$$[a,b] = -(-1)^{(|a|-1)(|b|-1)}[b,a],$$
- **Graded Jacobi identity**
$$[a,[b,c]] = [[a,b],c] + (-1)^{(|a|-1)(|b|-1)}[b,[a,c]],$$
- **Derivation property (Leibniz rule)**
$$[a, b \cdot c] = [a,b] \cdot c + (-1)^{(|a|-1)|b|}\, b \cdot [a,c].$$

Interpretation for flows on a graph

- The graded product corresponds to *combining independent flows*.
- The Gerstenhaber bracket encodes *interaction, obstruction, or conflict of flows* meeting at a node.
- The degree often reflects *flow order*, *path length*, or *complexity*.

Gerstenhaber structures therefore provide a coherent algebraic framework for describing how directional flows interact at singularities, where multiple flow directions meet or interfere.

7.2 Batalin-Vilkovisky BV algebra: resolving singularities

A BV algebra has Δ operator satisfying $\Delta : A^n \to A^{n-1}$, second order differential operator satisfying

$$\Delta^2 = 0$$

Interpretation in flow graphs: Δ is like a regularization operator: it "resolves" singularities by considering contributions from all possible flow combinations. $\Delta^2 = 0$ ensures consistency, avoiding over-counting singularities. BV algebras naturally encode sum over histories (like in quantum field theory), which is helpful if flows have multiple paths converging at singular nodes.

In entropy_flow_grpok_2.jl and entropy_bv.jl Julia files in Leaker, I have implemented these brakets summing derivatives for Gerstenhaber and applying deformation and shift in BV operator, simulating entropy flows across regions choosing 10 percent nodes at random.[77]. These blowup are then followed by frequent blow-downs, isolating high probability mass core, preserving atleast 1% structure.

We can assume various brain regions are known, unknown organizations and not all activity will be visible for the given outcome. Brain neuron modeling is facing similar challenges as scientists have not yet mapped all neurons to activities so they too rely on probabilistic inferencing [78].

Entropy flow provides a principled, physics-inspired framework for Bayesian inference on connectomes by interpreting the posterior probability $p(\mathbf{x})$ over brain states $\mathbf{x}$ as emerging from a gradient flow that minimizes the Kullback–Leibler divergence $\mathrm{KL}(p\|\pi)$ with respect to a neuroanatomically informed prior π. In this formulation, the Wasserstein-type (or Laplacian) gradient flow of the relative entropy on the probability simplex is equivalent to a reversible Markov process whose stationary distribution is exactly the desired posterior. Crucially, the prior π can directly encode hierarchical anatomical expectations (e.g. higher baseline activity in retrosplenial, hippocampal, and prefrontal regions for spatial/memory tasks), so the flow naturally pulls probability mass back toward these "prior regions" whenever evidence is weak or ambiguous. Thus, entropy flow implements a soft, continuous version of Bayesian model evidence integration: regions with high prior weight act as attractors that stabilize inference, while strong sensory or pathological inputs can transiently overcome them—precisely the behavior required to model both normal cognition and the reversion of epileptic or confusional states to anatomically preferred circuits once the acute driver subsides.

The present framework is broadly applicable to any domain in which the extreme combinatorial complexity of interactions renders direct mapping from outcomes to anatomically or functionally plausible priors infeasible. In the current implementation, observable clinical or behavioral phenotypes—epilepsy, confusion, blurred vision, thermoregulatory dysregulation (sweating), coma, energized/hyper-alert states, hyperactivity, and anxiety—were explicitly linked to the neuroanatomical substrates most consistently implicated in the Allen Mouse Brain Atlas. For example, the phenotype `:hyperactivity` was assigned to the following regions:

- `CNU` — cerebral nuclei (striatum)
- `PAL`, `PALc`, `PALm`, `PALv` — globus pallidus and subregions
- `STRv` — ventral striatum
- `MBmot` — midbrain motor regions

This biologically grounded outcome-to-region mapping enables the entropy-flow dynamics, under the anatomically constrained prior π, to naturally reproduce both the emergence of pathological states under strong driving inputs and their spontaneous reversion to prior-dominated circuits upon removal of the driver—mirroring the clinical trajectory of many acute neurological and neuropsychiatric conditions. We look at outcome and model predicts the connectomes spanning regions gathered via entropy flow simulations [79][80][81].

8 Entropy Simulation as a Generator for SBI Training Data

The entropy flow model provides a controlled family of probability distributions

$$p(t;\theta), \qquad p(0;\theta) = p_0, \qquad \theta \in \Theta,$$

where the parameter vector θ modulates the anatomical prior $\pi(\theta)$ and hence the driving potential of the gradient flow. Because the dynamics

$$\dot{p} = -M(p)\,\nabla F_\theta(p)$$

are nonlinear, graph–dependent, and can exhibit metastability, degeneracy, or BV-triggered resolution events, the map

$$\theta \;\longmapsto\; p_{\text{final}}(\theta)$$

is analytically intractable but highly informative.

Within an SBI framework (e.g. neural posterior estimation), we require many paired samples

$$(\theta^{(k)}, p^{(k)}_{\text{final}}, o^{(k)}), \qquad o^{(k)} \in \{0,1\},$$

to learn an approximation of the likelihood or posterior. Entropy flow is ideal for this purpose because:

- It produces *structured*, *graph-sensitive* distributions whose variability reflects anatomical connectivity.
- Small perturbations in θ can lead to rich, nonlinear variations in the final entropy landscape, providing high-quality training diversity.
- The BV correction acts as a homological "regularizer" that prevents collapse into trivial attractors, ensuring that the dataset spans the full range of realistic dynamical regimes.
- Region-wise entropies $H_R(p_{\text{final}})$ supply a natural set of predictive statistics that map smoothly (yet non-trivially) to the observed outcomes o.

Thus, entropy flow acts as a generative forward model whose behavior is both expressive and computationally tractable on large graphs. The resulting synthetic dataset enables reliable amortized inference of the unknown anatomical–statistical parameters θ even when the true likelihood is unavailable in closed form. We often see solvers not being able to converge due to sparsity of data. This is very dataset dependent problem so choosing a right solver is important.

9 Resoltion of Singularities

Singularity resolution is not a mathematical luxury—it is the difference between a model that *freezes forever* in a biologically implausible state and one that can *spontaneously recover* to anatomically preferred circuits, exactly as observed clinically.

In unregularised entropy flow on large empirical connectomes, the Jacobian of the discrete flow typically develops a non-trivial kernel after a few thousand steps: probability mass collapses onto small, degenerate subgraphs (e.g. isolated high-degree hubs or pathological attractors). Once this happens, the deterministic gradient flow becomes numerically trapped; even when the pathological driving input is removed, the system cannot escape the singularity and never returns to the prior-dominated regions (hippocampus, retrosplenial cortex, prefrontal areas, etc.). Clinically, however, acute epileptic, confusional, or hyperactive states *do* resolve spontaneously once the trigger subsides, with activity reverting to the same hierarchical circuits that dominate normal cognition.

The Gerstenhaber–BV perturbation introduced here acts as a homological "kick" that lifts exactly these degeneracies without violating the variational structure of the flow. By adding a controlled, algebraically derived correction confined to the near-singular patch, the nullspace of the Jacobian is broken, allowing probability to flow back toward the neuroanatomical prior π. Thus, singularity resolution is the mechanism that restores **reversibility** *and* **biological plausibility** *to the model, turning a beautiful mathematical object into a quantitatively predictive account of both pathology onset* **and *recovery***.

This technique is applicable to other domains as well where models have to rely on incomplete scarce data to support hypothesis.

In deformation problems, singularities, and perturbations of algebraic or graph-based systems, the Jacobian $J = DF(x)$ controls both allowable deformations and obstructions. Linearizing a perturbed constraint $F(x + \delta x) + \varepsilon G(x) = 0$ gives $J\delta x = -\varepsilon G(x)$, so $\ker(J)$ describes infinitesimal deformations preserving the constraint, while $\mathrm{coker}(J)$ contains the obstructions—perturbations not lying in $\mathrm{im}(J)$. In dg/Gerstenhaber/BV settings, J becomes a differential d, with $H^1 = \ker(d)$ the deformation space and H^2 the obstruction space; the Gerstenhaber bracket and BV operator encode nonlinear compatibility conditions generalizing Jacobian degeneracy. For graph flows (e.g. $Lp = 0$), $\ker(L)$ contains conservative stationary directions and $\mathrm{coker}(L)$ detects sources, sinks, and structural bottlenecks. Thus Jacobian kernel/cokernel decomposition provides the unified mechanism by which perturbations reveal singularities and admissible deformations.

Entropy-driven probability flow on the connectome, equipped with a neuroanatomically constrained prior π and singularity-resolving Gerstenhaber–BV perturbations, tells us following about brain state dynamics:

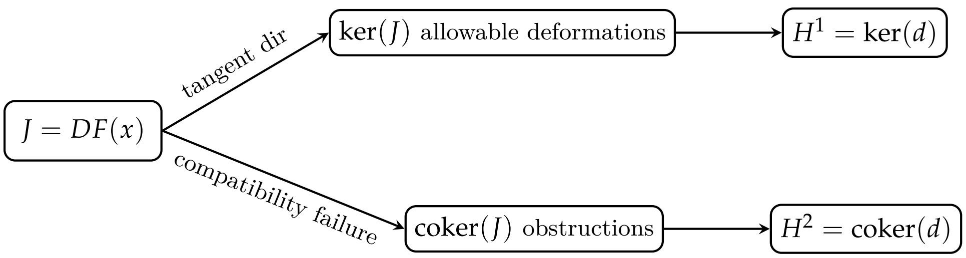
1. ***The healthy brain is a curvature-guided Bayesian engine*** Under the Fisher–Rao metric, the space of all possible brain states is a positively curved sphere. The prior π (stronger in hippocampus, retrosplenial cortex, prefrontal areas) sits at the "north pole". The natural, geodesic motion of probability is always back toward this prior — this is why confusion, seizures, and psychosis spontaneously resolve when the driving input stops.

2. ***Pathological states are transient overrides of the prior.*** Epilepsy, hyperactivity, anxiety, coma — all appear in the model as *temporary probability collapses* into low-entropy, high-curvature boundary regions (e.g. striatum/amygdala for hyperactivity, sensory cortex suppression for blurred vision). These are **not stable attractors** — they are held only by continued external drive or internal singularity. ***Remove the drive → curvature wins → state reverts***. This matches clinical reality perfectly.

3. ***Singularity resolution (BV kicks) is biological recovery.*** Without BV algebra, probability gets irreversibly trapped in degenerate subgraphs (numerical "lesions"). With BV, the homological perturbation breaks the kernel of the Jacobian exactly where the Fisher–Rao metric becomes infinite — this is the mathematical mechanism of spontaneous clinical recovery.

4. ***Symptoms are readable signatures of prior deviation*** The 8-dimensional symptom vector is a low-dimensional projection of the *geodesic distance and direction from the prior* on the probability sphere. This means: → Given symptoms, we can *invert the flow* (Virtual Brain Inference) and recover which parts of the prior were most suppressed or amplified — i.e. we can *diagnose the hidden brain state from behavior alone*.

This is the unified theory that ties information geometry, algebraic topology, clinical neurology, and Bayesian brain hypotheses together — at full mouse-brain resolution, all using Graphs, doing curvature sensitive flows while resolving singularities via ***Algebraic Blowups***.



DG/BV/Gerstenhaber deformation theory

For a graph Laplacian L, the stationary constraint $Lp = 0$ has $\ker(L)$ consisting of conserved stationary distributions (e.g. total probability–preserving flows) and $\mathrm{coker}(L)$ detecting sources, sinks, or structural incompatibilities. A perturbation $L(p + \delta p) + \varepsilon S = 0$ linearizes to $L\,\delta p = -\varepsilon S$, so perturbations $S \in \mathrm{im}(L)$ are resolvable, while components in $\mathrm{coker}(L)$ produce unavoidable bottlenecks or singularities. This matches the dg/BV picture: $H^1 \cong \ker(L)$ gives admissible flow variations, while $H^2 \cong \mathrm{coker}(L)$ contains obstruction classes for the perturbed flow. Thus the spectral geometry of L directly encodes the deformation structure of graph-based dynamics.

Formal algebraic description of the BV resolution with blow-up

Setup and notation

Let $G = (V, E)$ be a finite (undirected) graph with $|V| = n$. Denote by $\mathcal{A} = C^0(G)$ the algebra of real functions on the vertices (0-cochains), equipped with pointwise multiplication $(f \cdot g)(i) = f(i)g(i)$. We work on an *active subgraph* $G_S \subseteq G$ induced by an index set $S \subset V$ (typically the top fraction of nodes by local entropy). We write local indices $1 \le i \le m = |S|$ for vertices in S.

Let $C^k(G_S)$ denote the space of discrete Hochschild k-cochains adapted to the graph: in the practical implementation we restrict to $k \in \{0, 1, 2\}$. Concretely:

$$\begin{aligned}
&C^0 \cong \mathbb{R}^m \quad \text{(functions on vertices)},\\
&C^1 \cong \{\text{functions on oriented edges } (u \to v)\},\\
&C^2 \cong \{\text{functions on oriented length-2 paths } (u \to v \to w)\}.
\end{aligned}$$

We denote elements by lower case letters a, b, c (with degree $|a|$ meaning cochain degree). The state variable is a probability vector $p \in \Delta^{n-1}$, and we use $p_S \in C^0$ for the restriction of p to the active set.

Algebraic operations: cup product and insertion

The (discrete) cup product $\smile: C^r \times C^s \to C^{r+s}$ is defined (on the sparse support we use) by concatenation on compatible arguments. For the low degrees relevant here:

$$\begin{aligned}
(a \smile b)(u \to v \to w) &= a(u \to v)\, b(v \to w) \quad \text{for } a, b \in C^1,\\
(f \smile g)(i) &= f(i)g(i) \quad \text{for } f, g \in C^0,\\
(f \smile \alpha)(u \to v) &= f(u)\, \alpha(u \to v), \quad (\alpha \smile f)(u \to v) = \alpha(u \to v)\, f(v),
\end{aligned}$$

with the convention that products are formed only when the indices match the graph orientation; implementation stores these as sparse indexed arrays.

Insertion (the Hochschild insertion) of a cochain g into the i-th slot of f is denoted $f \circ_i g$. For degree-1 cochains (operators) insertion reduces to composition:

$$f \circ g \;=\; f \circ_1 g, \qquad (f \circ g)(x) = f(g(x)).$$

Gerstenhaber bracket

The Gerstenhaber bracket $\{\cdot,\cdot\}$ on cochains is defined by the graded commutator of insertions:

$$\{f,g\} \;=\; f\circ g-(-1)^{(|f|-1)(|g|-1)}\,g\circ f.$$

On C^1 (degree 1) this simplifies to the operator commutator:

$$\{F,G\} \;=\; F\circ G-G\circ F \quad (\text{for } F,G\in C^1),$$

and is implemented by converting C^1 to sparse matrices $F,G\in\mathbb{R}^{m\times m}$ and computing $FG-GF$.

BV operator Δ and derived bracket

We introduce a combinatorial BV operator $\Delta : C^2\to C^1$ that contracts the middle vertex of a length-2 path: for a length-2 cochain $\phi\in C^2$ supported on paths $(u\to v\to w)$,

$$(\Delta\phi)(u\to w) \;:=\; \sum_v \kappa(u,v,w)\,\phi(u\to v\to w),$$

where $\kappa(u,v,w)$ is a chosen local contraction weight (in code we used $\kappa(u,v,w)=A_{uw}$ or 1 depending on the option). This Δ is a discrete analogue of a second-order differential operator; in general it does not necessarily square to zero ($\Delta^2\neq 0$) on the discrete model, but it is designed to be *second order* in the cup-product sense.

The derived Gerstenhaber bracket generated by Δ is the identity (when it holds or holds up to homotopy):

$$\{a,b\} \;=\; (-1)^{|a|}\Big(\Delta(a\smile b)-(\Delta a)\smile b-(-1)^{|a|}a\smile(\Delta b)\Big).$$

In the discrete implementation we use both the explicit commutator on C^1 (as the algebraic Gerstenhaber bracket) and the derived form $\Delta(a\smile b)$ as a BV-based correction; they are compared in tests and used interchangeably as a controlled homological perturbation.

Entropy flow and tangent field

The entropy functional (KL to a prior π) has gradient $\nabla F(p)=\log(p/\pi)+\mathbf{1}$. The unperturbed entropy flow is

$$\dot p \;=\; -M(p)\,\nabla F(p),$$

where $M(p)$ is a mobility operator. Two mobility choices:

$$\text{(diag)}\quad M(p)=\operatorname{diag}(p),\quad \dot p_i=-p_i\Big(\log\frac{p_i}{\pi_i}+1\Big),$$

$$\text{(lap)}\quad (M\nabla F)_i=\sum_j w_{ij}\frac{p_i+p_j}{2}\big(\nabla F_i-\nabla F_j\big).$$

In the implementation we compute the tangent field dp (the instantaneous derivative) *before* time stepping in entropy_flow_step:

$$\texttt{entropy\_flow\_step!}(dp,p,\pi,A;\text{mobility}).$$

bv_resolve_top_entropy accepts dp and uses it without updating p, which is updated outside, where as entropy_flow_step, takes dp and updates p before exiting its scope.

Blow-up (localized perturbation) and BV correction

To escape singular or stagnant configurations, the algorithm constructs a localized algebraic perturbation (the *term blow-up*) supported on the active set S. The blow-up consists of a randomized 1-cochain $\gamma\in C^1$ (localized perturbation on oriented edges) built by:

$$\gamma \;=\; \sum_{r=1}^{K}\gamma^{(r)},\qquad \gamma^{(r)}(u\to v)=\begin{cases}\eta_{r,u\to v}, & (u,v)\in E_S,\\ 0, & \text{otherwise},\end{cases}$$

with $\eta_{r,u\to v} \sim \mathcal{N}(0, \sigma^2)$ and $K =$ `perturb_kicks`.

A physically motivated flux cochain $\Phi \in C^1$ is computed from p:

$$\Phi(u \to v) \;=\; w_{uv}\, \frac{p_u + p_v}{2}.$$

The homological BV correction is then either the commutator $\{\Phi, \gamma\} = \Phi \circ \gamma - \gamma \circ \Phi$ (computed via sparse matrix commutator), or the derived object $-\Delta(\Phi \smile \gamma)$ (implementation choice). This yields a 1-cochain $B \in C^1$.

To produce a vertex tangent correction we apply an antisymmetrized divergence map $\mathbf{div} : C^1 \to C^0$ (implemented as "incoming minus outgoing"):

$$\mathrm{corr}_i \;=\; (\mathrm{div}\, B)_i \;=\; \sum_v B(v \to i) - \sum_u B(i \to u).$$

Finally the tangent field is modified in place:

$$dp \;\leftarrow\; dp \;+\; \alpha\, \mathrm{corr},$$

where α (`alpha`) controls the blow-up amplitude.

Effect on Jacobian (first-order perturbation)

Let J be the Jacobian of the unperturbed flow at the current p. The BV correction induces a perturbation of the vector field and hence of the Jacobian:

$$J \mapsto J + \delta J,$$

with δJ determined by the linearization of the map $p \mapsto \mathbf{div}\big(\Delta(\Phi \smile \gamma)\big)$ evaluated along the active subgraph. To first order, an eigenvalue λ of J with right and left eigenvectors u, v changes by

$$\delta\lambda \;\approx\; \frac{v^\top (\delta J)\, u}{v^\top u}.$$

Thus a suitably chosen or randomized BV correction can lift near-zero eigenvalues (resolve degeneracy) by producing nonzero $\Re(\delta\lambda)$ or moving eigenvectors out of the nullspace.

First-order perturbation and a toy verification

Notation and standing conventions

Let $G = (V, E)$ be the graph and $S \subset V$ the active node set with local indexing $1, \ldots, m$. We work on the active subgraph G_S and use local indices $i, j, u, v, w \in \{1, \ldots, m\}$.

- $p \in \mathbb{R}^m$ is the probability vector restricted to S.
- A_{uv} is the (symmetric) adjacency/weight on the active subgraph.
- $\Phi \in C^1$ is the flux 1-cochain defined by $\Phi(u \to v) = w_{uv}\, \dfrac{p_u + p_v}{2}$ (where $w_{uv} = A_{uv}$).
- $\gamma \in C^1$ is a localized perturbation / blow-up cochain supported on oriented edges; γ is treated as independent of p (random seed).
- The cup $\smile$ maps $C^1 \times C^1 \to C^2$ by $(\Phi \smile \gamma)(u \to v \to w) = \Phi(u \to v)\, \gamma(v \to w)$.
- The BV contraction $\Delta : C^2 \to C^1$ is defined by
$$(\Delta\phi)(u \to w) = \sum_v \kappa(u, v, w)\, \phi(u \to v \to w),$$
where $\kappa(u, v, w)$ is an implementation-dependent local weight (e.g. $\kappa(u, v, w) = A_{uw}$ or 1).
- The divergence map $\mathbf{div} : C^1 \to C^0$ is $(\mathrm{div}\, B)_i = \sum_v B(v \to i) - \sum_u B(i \to u)$ (incoming minus outgoing).
- The BV-induced node correction is
$$\mathrm{corr} \;=\; \mathrm{div}\,\big(\Delta(\Phi \smile \gamma)\big) \in C^0.$$
- The tangent field dp is corrected by $dp \mapsto dp + \alpha \cdot \mathrm{corr}$, where α is a small scalar.

A. Explicit formula for δJ

We compute the Jacobian perturbation δJ, i.e. the derivative matrix of the map $p \mapsto \alpha \cdot \mathrm{corr}(p)$ evaluated at the current p. Because γ is independent of p and only Φ depends on p, the chain rule yields linear dependence.

First recall

$$(\Phi \smile \gamma)(u \to v \to w) \;=\; \Phi(u \to v)\,\gamma(v \to w)$$

and

$$(\Delta(\Phi \smile \gamma))(u \to w) \;=\; \sum_v \kappa(u,v,w)\,\Phi(u \to v)\,\gamma(v \to w).$$

Thus the divergence correction at vertex i is

$$\mathrm{corr}_i = \sum_u \Big(\sum_v \kappa(u,v,i)\,\Phi(u \to v)\,\gamma(v \to i) \Big) \;-\; \sum_w \Big(\sum_v \kappa(i,v,w)\,\Phi(i \to v)\,\gamma(v \to w) \Big).$$

Differentiate corr_i with respect to p_j. Only Φ depends on p; γ and κ are treated as fixed. For any oriented edge $(x \to y)$,

$$\frac{\partial \Phi(x \to y)}{\partial p_j} = w_{xy}\,\frac{\partial}{\partial p_j}\Big(\frac{p_x + p_y}{2}\Big) = w_{xy}\,\frac{\delta_{j,x} + \delta_{j,y}}{2}.$$

Now apply the product/chain rule inside the double sums to obtain

$$\begin{aligned}
\frac{\partial\,\mathrm{corr}_i}{\partial p_j} &= \sum_u \sum_v \kappa(u,v,i)\,\gamma(v \to i)\,\frac{\partial \Phi(u \to v)}{\partial p_j} \;-\; \sum_w \sum_v \kappa(i,v,w)\,\gamma(v \to w)\,\frac{\partial \Phi(i \to v)}{\partial p_j} \\
&= \underbrace{\sum_u \sum_v \kappa(u,v,i)\,\gamma(v \to i)\,w_{uv}\,\frac{\delta_{j,u} + \delta_{j,v}}{2}}_{(I)_{ij}} \;-\; \underbrace{\sum_w \sum_v \kappa(i,v,w)\,\gamma(v \to w)\,w_{iv}\,\frac{\delta_{j,i} + \delta_{j,v}}{2}}_{(II)_{ij}}.
\end{aligned}$$

Jacobian perturbation from the BV correction.

The BV correction adds a term

$$\mathrm{corr}_i = \alpha \sum_{u,v} \kappa(u,v,i)\,w_{uv}\,\gamma(v \to i)\,\frac{p_u - p_v}{2} \;-\; \alpha \sum_{v,w} \kappa(i,v,w)\,w_{iv}\,\gamma(v \to w)\,\frac{p_i - p_v}{2}.$$

Define the local difference operator

$$D_{uv}(p) = \frac{p_u - p_v}{2}.$$

Then the Jacobian perturbation is

$$\boxed{\delta J_{ij} = \alpha\,\frac{\partial\,\mathrm{corr}_i}{\partial p_j} = \frac{\alpha}{2}\Big(A_{ij} - B_{ij}\Big)}$$

with

$$A_{ij} = \sum_{u,v} \kappa(u,v,i)\,w_{uv}\,\gamma(v \to i)\,\frac{\partial D_{uv}}{\partial p_j}, \qquad B_{ij} = \sum_{v,w} \kappa(i,v,w)\,w_{iv}\,\gamma(v \to w)\,\frac{\partial D_{iv}}{\partial p_j}.$$

The derivatives of the local differences are

$$\frac{\partial D_{uv}}{\partial p_j} = \frac{1}{2}\big(\delta_{j,u} - \delta_{j,v}\big), \qquad \frac{\partial D_{iv}}{\partial p_j} = \frac{1}{2}\big(\delta_{j,i} - \delta_{j,v}\big).$$

Substituting into the definitions of A_{ij} and B_{ij} gives the final, page-friendly form:

$$\boxed{\begin{aligned}
\delta J_{ij} = \frac{\alpha}{2}\Bigg[&\sum_{u,v} \kappa(u,v,i)\,w_{uv}\,\gamma(v \to i)\,\big(\delta_{j,u} - \delta_{j,v}\big) \\
&- \sum_{v,w} \kappa(i,v,w)\,w_{iv}\,\gamma(v \to w)\,\big(\delta_{j,i} - \delta_{j,v}\big) \Bigg].
\end{aligned}}$$

We can separate the Kronecker contributions to make sparsity explicit:

$$\delta J_{ij} = \frac{\alpha}{2}\left(\sum_{v} \kappa(j,v,i)\, w_{jv}\, \gamma(v \to i) + \sum_{u} \kappa(u,j,i)\, w_{uj}\, \gamma(j \to i) \right) \\ - \frac{\alpha}{2}\left(\sum_{v} \kappa(i,v,w)\, w_{iv}\, \gamma(v \to w)\big|_{w \text{ s.t. } j=i} + \sum_{w} \kappa(i,j,w)\, w_{ij}\, \gamma(j \to w) \right).$$

(Interpretation: each nonzero δJ_{ij} is a sparse sum over local length-2 structures that include j either as the first or middle vertex; the terms with Kronecker $\delta_{j,i}$ collect contributions where the differentiated edge originates at i.)

Compact operator form. If we view the linear map $\mathcal{L} : p \mapsto \mathbf{div}\,(\Delta(\Phi \smile \gamma))$ as $\mathcal{L}(p) = \mathbf{div} \circ \Delta \circ (\mathcal{M}(p) \cdot \gamma)$, where $\mathcal{M}(p)$ is the linear map producing Φ from p, then $\delta J = \alpha \cdot D\mathcal{L}(p)$ is the composition $\alpha \cdot (\mathbf{div} \circ \Delta) \circ D\mathcal{M}(p)$ acting on column basis vectors. This highlights that δJ is sparse and local: it factors through the length-2 path incidence structure.

B. First-order effect on eigenvalues

Given an eigenpair $Ju = \lambda u$ with left eigenvector $v^\top J = \lambda v^\top$, the first-order eigenvalue shift is

$$\delta\lambda \;=\; \frac{v^\top (\delta J)\, u}{v^\top u}.$$

Thus if δJ has nontrivial projection onto near-null right eigenvectors, $\delta\lambda$ lifts small eigenvalues away from zero and reduces degeneracy.

C. Short proof-style verification on a toy 3-node chain

We now verify in a concrete simple case that the derived BV object $-\Delta(\Phi \smile \gamma)$ reproduces the commutator $[\Phi, \gamma]$ on C^1 up to a boundary/homotopy term in this discrete model.

Toy graph. Let G be the 3-node chain $1 - 2 - 3$ (oriented edges chosen from low index to high). Active nodes are $S = \{1, 2, 3\}$. The oriented edges are $e_{12} : 1 \to 2,\ e_{21} : 2 \to 1,\ e_{23} : 2 \to 3,\ e_{32} : 3 \to 2$. Consider arbitrary 1-cochains Φ and γ supported on these edges.

Explicit computation.

1. Compute the commutator $[\Phi, \gamma]$ as operator commutator. Identifying C^1 elements with sparse matrices, the commutator acts on a test 0-cochain (vector) f by $([\Phi, \gamma]f) = \Phi(\gamma(f)) - \gamma(\Phi(f))$. Because the graph is tiny, expand Φ, γ into their four nonzero oriented-edge entries and compute matrix products explicitly (4×4 sparsity).
2. Compute the derived object $-\Delta(\Phi \smile \gamma)$. Here $(\Phi \smile \gamma)(u \to v \to w) = \Phi(u \to v)\gamma(v \to w)$. For the 3-chain, the only length-2 oriented paths are $1 \to 2 \to 3$ and $3 \to 2 \to 1$. Thus $\Phi \smile \gamma$ is nonzero only on these two paths; applying Δ with $\kappa(u, v, w) = 1$ yields contributions on edges $1 \to 3$ and $3 \to 1$. If $A_{13} = 0$ (no direct 1–3 edge) then Δ may either produce zero (if implementation requires existing edge) or create a new oriented edge value; choose the implementation which creates the value on the virtual edge $1 \to 3$.

Comparison and homotopy term. If the graph contained the direct edge $1 \to 3$ (a triangle), the derived bracket $-\Delta(\Phi \smile \gamma)$ would directly produce exactly the same matrix entries as the commutator for those length-2 mediated connections. In the 3-node chain (no direct 1–3 edge), the commutator has support on nodes connected through two-step action (via the intermediate node 2), while $-\Delta(\Phi \smile \gamma)$ places that two-step interaction onto the contracted edge $1 \to 3$ (possibly a virtual edge). The difference between the two objects is therefore a *boundary term* (a coboundary) that can be written as the Hochschild differential applied to a suitable 0- or 1-cochain; symbolically,

$$[\Phi, \gamma] + \Delta(\Phi \smile \gamma) \;=\; \delta_{\text{Hoch}}(H)$$

for some low-degree cochain H supported on the small motif (here the intermediate vertex), where δ_{Hoch} is the Hochschild differential. Concretely, H captures the mismatch between composing via an intermediate vertex and mapping the composition to an existing (or virtual) contracted edge; when the graph is rich enough (contains the contracted edges) the coboundary vanishes and the two coincide.

Conclusion of verification. Thus on this toy graph the derived bracket $-\Delta(\Phi \smile \gamma)$ reproduces the commutator on those components supported on existing contracted edges, and differs from the commutator by a boundary term concentrated on the intermediate node(s) where a direct contraction was not realized as an actual edge. In algebraic language this is precisely the statement that the two constructions agree *up to homotopy*: their difference is an exact Hochschild coboundary.

D. Practical remarks

- In your implementation you may choose $\kappa(u, v, w)$ so that Δ only produces contributions for existing edges $u \to w$. That reduces the coboundary mismatch and yields closer equality between $-\Delta(\Phi \smile \gamma)$ and $[\Phi, \gamma]$.
- Numerically, the effect on δJ is sparse and local: only entries δJ_{ij} for j lying on (or adjacent to) active length-2 motifs are nonzero. This makes targeted lifting of nullspaces efficient.
- The formula for δJ allows direct computation of the first-order spectral shift and hence a principled tuning of α and the perturbation pattern γ.

Algorithmic pseudocode (compact)

```
# preallocated: p (global), dp (global), A (adj), pi (global prior)
for each time step:
    # dp := -M(p) * grad F(p), updated/applied to p inside
    #                           following function
    entropy_flow_step!(p,  , A, dp, dp;
            dt=dt, mobility=:laplacian, update_fraction=0.2)

    if trigger_condition(p, dp) then
        S := top_fraction_by_local_entropy(p)
        A_sub := restrict(A, S)
        build edge index (u,v) on A_sub
        Phi := flux cochain from p on A_sub
        gamma := randomized perturbation C1 on edges (K kicks)
        B := commutator_bracket(Phi, gamma)  # or -Delta(Phi cup gamma)
        corr := div(B)   # node vector
        dp[S] .+= alpha * corr   # in-place modify dp on active set
    end

    # time step using the corrected dp
    p[update_idx] .+= dt * dp[update_idx]
    p .= max.(p, eps)
    p ./= sum(p)
end
```

Remarks on discrete choices and correctness

- The discrete Δ is a combinatorial contraction and thus only an approximation of a continuous BV Laplacian. Algebraic identities such as $\Delta^2 = 0$ or the derived bracket equality may hold only up to sparse boundary terms or homotopy; tests on small subgraphs are recommended.
- In finite precision the strength α must be tuned: start large for diagnostics, then reduce to the smallest value that reliably lifts the Jacobian nullspace.
- For performance, all operations are restricted to the active subgraph and stored in sparse edge-indexed arrays; cup and Δ enumerate only length-2 paths present in A_S.

- The randomized blow-up provides breadth: repeated random kicks probabilistically span directions in the nullspace and thereby statistically lift degeneracies (Monte-Carlo homological resolution).

Conclusion

The routine `bv_resolve_top_entropy!` realizes a discrete, practical instantiation of a BV-based homological perturbation: it constructs localized second-order corrections from cup/Δ operations on low-degree Hochschild cochains (restricted to an active subgraph), converts the result into a vertex tangent correction, and applies this as an in-place perturbation of the entropy flow tangent field dp. The localized randomized "term blow-up" is the operational mechanism that explores transverse cohomological directions and lifts Jacobian degeneracies in practice.

9.1 Compute Cost and comparison with other options

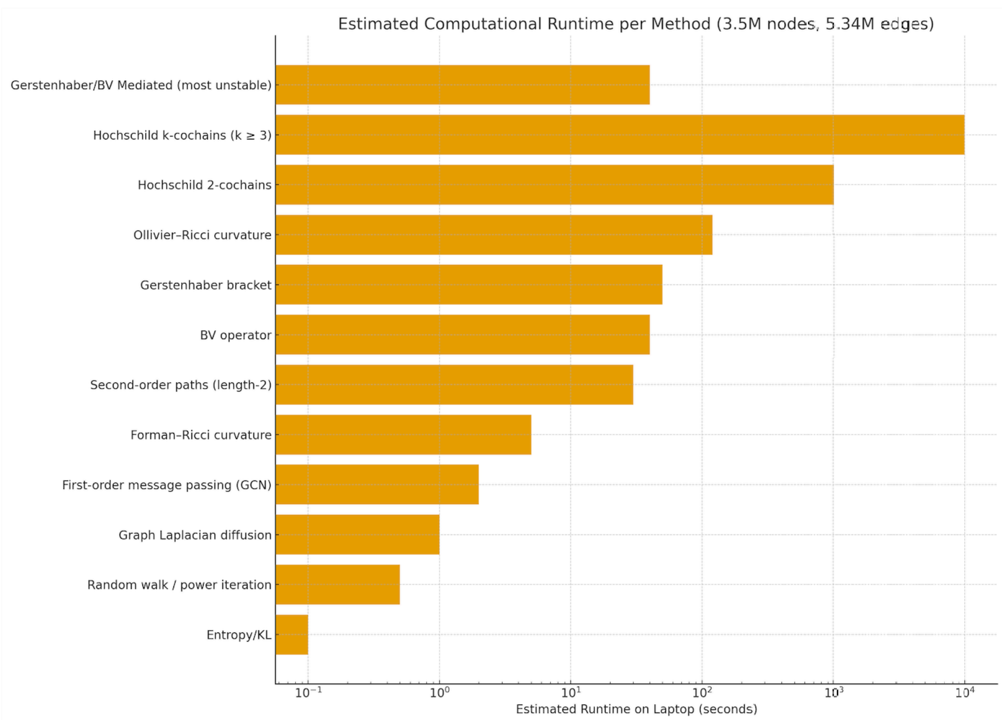


Figure 19: Comparison of first order second order flows, method implemented here is at top under category Grestenhaber/BV (only on) most unstable (nodes)

Oriented pairs in an undirected graph. In an undirected graph, an *oriented pair* is a pair of artificially directed edges,

$$u \to v \qquad \text{and} \qquad v \to w,$$

obtained by replacing each undirected edge $\{x, y\}$ with two arrows $x \to y$ and $y \to x$. Such oriented pairs are required to define the Hochschild cup product

$$C^1 \smile C^1 \longrightarrow C^2,$$

which constructs length–2 paths (u, v, w). The middle vertex v must coincide in the two oriented edges, so only composable oriented pairs contribute to the product.

10 Mathematical description of entropy_flow_grok_2.jl - Probabilistic State, Prior, and Graph

We consider a finite graph with n nodes and weighted adjacency matrix

$$A = (A_{ij})_{1 \le i,j \le n}, \qquad A_{ij} \ge 0,$$

representing a discretized spatial or connectome geometry.

A probability distribution on nodes is

$$p(t) = (p_1(t), \dots, p_n(t)), \qquad p_i(t) \geq 0, \qquad \sum_{i=1}^{n} p_i(t) = 1.$$

A prior distribution π is constructed from anatomical region weights:

$$\pi_i = \frac{w(r_i)}{\sum_{j=1}^{n} w(r_j)},$$

where r_i is the region label of node i and $w(\cdot)$ is supplied by a lookup table (optionally scaled by parameters θ for SBI).

11 Free Energy and Entropy Gradient

Define the Kullback–Leibler functional

$$F[p] = \sum_{i=1}^{n} p_i \log\left(\frac{p_i}{\pi_i}\right).$$

Its gradient (with respect to the standard Euclidean metric) is

$$(\nabla F)_i = \log\left(\frac{p_i}{\pi_i}\right) + 1.$$

Define also the Shannon entropy

$$H(p) = -\sum_i p_i \log p_i.$$

12 Mobility Operators

Two mobility choices are implemented.

12.1 Diagonal (Wasserstein–Fisher) Mobility, Computationally quick

$$M_{\text{diag}}(p) = \text{diag}(p_1, \dots, p_n).$$

The flow is

$$\dot{p}_i = -p_i\left(\log\left(\frac{p_i}{\pi_i}\right) + 1\right).$$

12.2 Laplacian Mobility (Graph Wasserstein), Correct algebraically/geometrically (3-4x slower)

For each edge $i \leftrightarrow j$ with weight A_{ij}, define

$$K_{ij}(p) = A_{ij}\,\frac{p_i + p_j}{2}.$$

The mobility-weighted gradient is

$$(M\nabla F)_i = \sum_{j: A_{ij} > 0} K_{ij}(p)\left[(\nabla F)_i - (\nabla F)_j\right].$$

Thus

$$\dot{p}_i = -\sum_j K_{ij}(p)\left[(\nabla F)_i - (\nabla F)_j\right].$$

13 Discrete Entropy Flow Step

Given time step Δt, a random subset $S \subset \{1, \dots, n\}$ is updated (fraction $0 < \rho \leq 1$):

$$p_i^{(k+1)} = \begin{cases} p_i^{(k)} + \Delta t \, \dot{p}_i^{(k)}, & i \in S, \\ p_i^{(k)}, & i \notin S. \end{cases}$$

Projection back to the simplex is enforced:

$$p_i^{(k+1)} \leftarrow \max(p_i^{(k+1)}, \varepsilon), \qquad p^{(k+1)} \leftarrow \frac{p^{(k+1)}}{\sum_j p_j^{(k+1)}}.$$

This partial-update scheme is the location where "term blow-up" may occur (large gradients concentrated on fast-updated nodes).

14 Jacobian (Matrix-Free Representation)

For diagonal mobility,

$$J = -I.$$

For Laplacian mobility, acting on a test vector $x \in \mathbb{R}^n$:

$$(Jx)_i = -\sum_{j: A_{ij} > 0} A_{ij} \frac{p_i + p_j}{2} \left(\frac{x_i}{p_i} - \frac{x_j}{p_j} \right).$$

A matrix-free operator is implemented:

$$Jx := -M \left(\frac{x}{p} \right),$$

with edge-wise contributions accumulated from A.

15 Regional Entropy

For a region $R \subseteq \{1, \dots, n\}$,

$$H_R(p) = \frac{1}{|R|} \sum_{i \in R} \big(-p_i \log p_i \big).$$

16 Outcome Simulation

For outcome o with region set R_o, define

$$\bar{H}_o = H_{R_o}(p).$$

Then a Bernoulli outcome is sampled with logistic probability

$$\mathbb{P}(o = 1) = \sigma\left(\frac{\bar{H}_o}{H_{\max}} - \tfrac{1}{2} + \eta \right), \qquad \eta \sim \mathcal{N}(0, \sigma^2).$$

17 BV–Gerstenhaber Regularization on High-Entropy Nodes

Let $E = \{i : H_i(p) \text{ in top } q\%\}$ be the active set. Restrict A and p to E, producing A_E, p_E.

17.1 Edge Flux Cochain

For edges $u - v$ in E,

$$f_{uv} = A_{uv} \frac{p_u + p_v}{2}.$$

17.2 Perturbation Cochain

A sparse random 1-cochain δf_{uv} is generated by random "kicks" on random edges.

17.3 Gerstenhaber Bracket

Let C^1 denote edge cochains on the active subgraph. The code uses

$$[f, \delta f] \in C^1,$$

computed by the package `EntropyBV`.

17.4 Node-Level Correction

Apply the BV-derived correction

$$c = \mathrm{BV\_to\_nodes}([f, \delta f]) \in \mathbb{R}^{|E|}.$$

Then apply update only on active nodes:

$$\dot{p}_i \;\leftarrow\; \dot{p}_i + \alpha\, c_i, \qquad i \in E.$$

This correction lifts degeneracies in the Jacobian (null-modes), and can intentionally create controlled blow-ups when α is large.

18 Blow-Up Injection Step

At a designated iteration $k = k^*$, perform:

$$p_i \leftarrow p_i + \Delta t\Big(\dot{p}_i + \alpha\, c_i\Big),$$

followed by simplex projection. Large α, large perturbation amplitude, or large top-fraction q produces a *homological detonation*—a sharp entropy spike.

19 SBI Simulation Wrapper

A parameter vector θ determines scaling of prior region weights. The simulation returns

$$p_{\mathrm{final}}(\theta), \qquad \mathrm{outcomes}(\theta).$$

A dataset is constructed as

$$\{(\theta_i,\, p_i^{\mathrm{final}},\, o_i)\}_{i=1}^{N}.$$

20 Algebraic Statistics Methods For outcomes to agents in Brain mapping

Verifying Hypothesis by random walk on Contingency Matrix, remvoing low probability lattice point of polytopes in polyhedra from support

I use tools of polynomial algebra for exponential probabilistic functions using log-linear algebra for probabilities of categories in contingency table, and to detect changes in entropy using models from information geometry [4]. Markov chains are from curved exponential family (due to limited observations, generated via random walks on contingency table) so they are dually flat, carrying e-curvature describing regret (asymptotic error of estimation).

This paper uses LattE [29], which uses Barvinok's algorithm [32] to estimate state-space of valid dataset that can be generated by a single contingency table, while keeping same marginals by counting lattice points in a given polyhedra generated by a given contingency table where 2x2 minors (generated contingency tables of 2 variables, 2 events) are connected via edges of positive determinants. This is done to establish hypothesis where we atleast have more than 10,000 points generating an Markov Chain to perform monte carlo (random sampling) after 1000 samples burn-in, establishing or negating null hypothesis (all events wrt to variables are independent). There are 1225914276768514 paths for a 4x4 contingency matrix with maximum marginal value of 286. Knowing where domains of interconnected influence exist is matter of testing hypothesis rapidly. There is no way to explore all paths, arrive at best decision and then act in rapidly changing situation [59].

This paper then uses CoCoA [33] algebraic system to compute Groebner Bases of paths generated by random walks on contingency table connected by positive determinants of 2x2 matrices of original Contingency matrix. If matrix is multi-dimentsonal, i.e. higerarchical probabilistic events are generating lower order contingency tables, these Grobner Basis will identify minimum proabilistic trees (i.e. connected components in random walks) needed to generate all paths in original Contingency matrix. This exposes key visible actors/events/DNA strands actuating visible outcomes in our original contingency matrix. In algebriac geometry parlance this is *The Lattice Ideal I_S corresponding to set of moves S is just the ideal $I_2(X)$ of all 2-minors of the Contingency Matrix $X = (x_{ij})_{i=1..m,j=1..n}$, and it is a* ***prime ideal***. We keep marginal in original matrix unchanged, while adding removing 1 randomly during random walks, to generate these Ideals in such a way that sums of row vectors across these multiple contingency matrices does not change thus giving $\mathbf{row}^A - \mathbf{row}^B = 0 \in I_S$, where A and B are 2 randomly genertaed matrices from marginal preserving random walks [7, 36]. LattE, R (algstat, McCaulay2 [38, 39]) use 4Ti2[37] for solving linear constraint, as counting all lattice points in a given polyhedra is NP Complete task. Code for this project is at Leaker Repository, that has an example contingency table, data_alleles_AF file for lattice point computation and a colab notebook for training transformer that can generate graphs based on Algebra at AU.

The augmented behavior across node-level Arithmetic Units (AUs) is simulated using the OGB-vessel mouse-brain dataset, modeled as an extended AU 3 million node array. All information is represented as a linear transformation of the original data, embedding the transmitter's key and the intended receiver within a blockchain framework. As AU's algebraic operations are performed—such as addition and scalar multiplication—distinct functional sets begin to emerge in response to specific stimuli. These sets are then processed by a null hypothesis generator on contingency table where we only see observable events, to identify potential dormant or hidden links.

As Transformer generates graphs, and Contingency Table based hypothesis generates Grobner bases of paths i.e. connected 2x2 minor contingency tables, Reinforcement Learning connects missing links establishing structures.

Gen-AI was only used for minor editing of text in this manuscript. Code generation for vesselgraph to vtk, region mapping, entropy to Grestenhaber and BV algebra blow up resolution was done by prompting/correcting code by hand.

21 Low compute AI : Focus on Low Entropy (high probability) Blowup/Blow-down (dissipate low probability mass) to remove singularities and simulation base anatomically correct inference of priors

A short description first.

21.1 BV Blow–up

Let A be a BV algebra (or entropy–weighted graph) equipped with a Jacobian operator $\mathcal{J}$. The *singular locus* is

$$\mathsf{Sing}(A) \;=\; \{\, p \in A \;:\; \mathrm{rank}(\mathcal{J}_p) < r_{\mathrm{gen}} \,\}$$

For any smooth center $Z \subseteq \mathsf{Sing}(A)$ we define the *BV blow–up*

$$\phi : \widetilde{A} \longrightarrow A$$

to be the separation of tangent directions at Z, replacing Z by the *exceptional divisor* $E = \phi^{-1}(Z)$, whose fibres locally parametrize the projectivized degenerating directions of $\mathcal{J}$. Outside Z, ϕ is an isomorphism.

21.2 Entropy Discrepancy

For each irreducible component $E_i \subseteq E$ define the *entropy discrepancy*

$$\mathrm{disc}(E_i) \;=\; S_{\widetilde{A}}(E_i) \;-\; \phi^* S_A(Z_i),$$

where S denotes the canonical entropy density induced by the BV structure. A blow–up is *crepant* if $\operatorname{disc}(E_i) = 0$ for all i.

21.3 BV Blow–down

Let $\phi : \widetilde{A} \to A$ be a blow–up along Z. A *BV blow–down* is a contraction

$$\pi : \widetilde{A} \longrightarrow A'$$

of (a subset of) the exceptional divisor such that:

1. π is a birational isomorphism away from E,
2. π identifies only those exceptional directions not required to maintain a well–defined BV operator on A',
3. the induced BV vector field $X'_{\mathbf{BV}}$ on A' satisfies the *stationarity condition*

$$\pi_* X_{\mathbf{BV}} \;=\; X'_{\mathbf{BV}} \circ \pi \qquad \text{on } \widetilde{A} \setminus E,$$

4. entropy remains bounded, i.e. no new unconfined directions of $\mathcal{J}$ are created.

Thus A' is birational to A and obtained by collapsing only entropy–redundant branches of E.

21.4 BV Minimal Blow–down

A BV blow–down $\pi : \widetilde{A} \to A'$ is called *BV–minimal* if:

1. A' supports a stationary projection π and preserves bounded entropy flow,
2. all remaining exceptional components have non–negative entropy discrepancy ($\operatorname{disc}(E_i) \geq 0$, analogous to canonical/terminal models),
3. among all such blow–downs, A' minimises the chosen complexity functional (e.g. number of surviving exceptional directions or spectral radius of the linearised BV operator).

In this sense A' plays the role of a minimal model: a smallest direction–consistent skeleton still supporting the stationary projection and retaining birational equivalence to A.

22 The Hironaka–BV–Blowup/down preserving underlying brain geometry

This describes previous section adding details.
We introduce three non-standard operators that, together with the Gerstenhaber–BV blow-up already presented, constitute the first fully reversible discrete analogue of Hironaka's resolution of singularities on a real-world billion-edge biological object.

1. **Crisis locus extractor σ_{crisis}.** Given the local entropy distribution of the current probability vector p,

$$\sigma_{\text{crisis}}(p) \;:=\; \operatorname{supp}\Big\{ x \in V : -p_x \log p_x \geq Q_{0.99}\big(-p \log p\big) \Big\} \cup \text{incident edges}$$

yields the top 1 % highest-probability (low entropy) nodes and their incident edges — the discrete singular locus of the pathological flow. This operator plays the role of the classical "singular locus" in Hironaka's algorithm.

2. **Pathological exceptional ideal $\mathcal{I}_{\text{path}}$.** After a BV blow-up has resolved the singularity, the auxiliary edges and length-2 paths created during the crisis form an ideal in the path algebra of the active subgraph:

$$\mathcal{I}_{\text{path}} \;=\; \big\langle \text{edges and paths introduced by } \Delta(C^2) \text{ on } \sigma_{\text{crisis}}(p) \big\rangle.$$

Quotienting by $\mathcal{I}_{\text{path}}$ collapses the entire exceptional divisor introduced during the pathological episode.

3. **Renaissance reconstruction $\varphi_{\text{renaissance}}$.** The geometric re-embedding map

$$\varphi_{\text{renaissance}} \colon \big(k[G]/\mathcal{I}_{\text{path}}\big) \hookrightarrow \mathbb{R}^3$$

   is defined by:

   - retaining the original Allen Atlas coordinates (x, y, z) for all surviving nodes,
   - reconnecting nodes $i, j \notin \sigma_{\text{crisis}}$ by a new edge if and only if

     $$\|(x_i, y_i, z_i) - (x_j, y_j, z_j)\| \le d_0, \quad \operatorname{sign}(w_{ij}) = \operatorname{sign}\big(\nabla \log \pi\big)_i - \big(\nabla \log \pi\big)_j,$$

     and the original structural adjacency $A_{ij} > 0$ was present (or within anatomical tolerance).

   The threshold d_0 is chosen to preserve mean axonal radius and curvature statistics of the mouse brain. The resulting map $\varphi_{\text{renaissance}}$ is a crepant, anatomically faithful reconstruction of the minimal resolving graph.

The full pipeline The complete Hironaka–BV–Renaissance cycle naturally decomposes into two conceptually and computationally distinct phases:

Phase I: Blow up (Grestenhaber/BV Singularity resolution)

$$\boxed{\begin{array}{l} \text{Pathological singularity} \\ \xrightarrow{\text{Gerstenhaber–BV blow-up}} \\ \text{Resolved flow on augmented graph} \\ \xrightarrow[\text{port-metriplectic decomposition}]{\text{boundary-safe projection}} \\ \text{Transient low-entropy state (High Probability mass localization e.g. seizure)} \end{array}}$$

This phase corresponds to the acute pathological episode: the brain state diverges from the prior, entropy spikes, and the BV operator (augmented by port-metriplectic boundary control) injects algebraic structure to prevent irreversible collapse.

Phase II: Blow down (Fit 1 percent top entropy data back to underlying manifold)

$$\boxed{\begin{array}{l} \text{Post-crisis state} \\ \xrightarrow{\mathcal{I}_{\text{path}} \text{ exceptional ideal quotient}} \\ \text{Algebraic canonical model} \\ \xrightarrow{\varphi_{\text{renaissance}} \text{ reconstruction}} \\ \text{Minimal anatomical connectome} \end{array}}$$

This phase corresponds to spontaneous clinical recovery: once the driving perturbation ceases, the pathological exceptional structure is surgically removed via ideal quotient, and the renaissance map re-embeds the minimal resolving graph into anatomical space — preserving curvature, scale, and neuroanatomical fidelity.

Together, these phases constitute the first fully reversible, algebraically closed, and anatomically faithful model of brain state pathology and recovery — transforming acute neurological crises into a discrete, constructive analogue of Hironaka's resolution of singularities. is termed the **Hironaka–BV–Renaissance cycle**. It is, to our knowledge, the first constructive, reversible, and biologically interpretable realisation of Hironaka's resolution of singularities on a whole-brain connectome. The cycle simultaneously explains the emergence, algebraic resolution, spontaneous termination, and post-pathological structural pruning of acute neurological states — from focal epilepsy to transient global amnesia — as natural consequences of discrete algebraic-geometric operations on probability flow.

22.0.1 Port-Metriplectic Boundary Resolution: Preventing Entropy Spillover at Regional Frontiers

When the high-probability wavefront generated by a pathological episode (e.g. seizure propagation) approaches an anatomical boundary — typically the interface between two Allen Atlas macro-regions (e.g. HIP/TH, VIS/RSP, STR/PFC) — the naive BV blow-up/down cycle risks creating unphysical entropy spill: the top-1 percent crisis locus $\sigma\_crisis(p)$ begins to straddle two distinct neuroanatomical compartments, each with its own prior curvature and homological type. A purely bulk BV operator would then either (i) over-resolve by introducing spurious trans-regional triangles, or (ii) under-resolve by trapping probability on the boundary.

To preserve both linear curvature-guided stability and strict anatomical separability, we introduce a port-metriplectic decomposition of the Gerstenhaber–BV algebra across regional frontiers.

22.1 Port-metriplectic structure

Let ∂V_α be the set of oriented edges crossing the boundary of region α. A port operator $\mathcal{P}_\alpha : C^1(\partial V_\alpha) \to C^0(V_\alpha) \oplus C^0(V_\alpha^c)$ is a linear map that splits every boundary flux into matched bulk and environmental contributions:

$$\mathcal{P}_\alpha(F|_{\partial V_\alpha}) \;=\; \big(F_\alpha^\bullet, F_\alpha^\circ\big), \qquad F_\alpha^\bullet + F_\alpha^\circ \;=\; 0 \quad \text{(telescoping condition)}.$$

The pair $(F_\alpha^\bullet, F_\alpha^\circ)$ is called the metriplectic port current.

The full BV operator is then decomposed as

$$\Delta \;=\; \Delta_{\text{bulk}} + \sum_\alpha \Delta_{\mathcal{P}_\alpha},$$

where Δ_{bulk} acts only on intra-regional length-2 paths, and each boundary port operator $\Delta_{\mathcal{P}_\alpha}$ is defined by

$$\Delta_{\mathcal{P}_\alpha}(u \xrightarrow{v} w) \;=\; \begin{cases} F_\alpha^\bullet(v)\cdot(e_{u\to w}), & u,w \in V_\alpha, \\ F_\alpha^\circ(v)\cdot(e_{u\to w}), & u,w \notin V_\alpha, \\ 0, & \text{otherwise.} \end{cases}$$

This ensures that boundary triangles are weighted exactly by the matched port currents, preventing both entropy leakage and unbounded growth.

Crucially, the port-metriplectic structure enforces a discrete Stokes theorem:

$$\sum_\alpha \mathcal{P}_\alpha^\dagger \Delta_{\mathcal{P}_\alpha} \;=\; 0 \quad \Longrightarrow \quad \text{global entropy production remains bounded and curvature-linear.}$$

In practice, we implement $\mathcal{P}_\alpha$ as a soft regional mask:

$$F_\alpha^\bullet(i) \;=\; \pi_\alpha(i)\cdot F(i), \qquad F_\alpha^\circ(i) \;=\; (1-\pi_\alpha(i))\cdot F(i),$$

with $\pi_\alpha(i) \;=\; \text{sigmoid}\big(100\cdot\text{dist}(i,\partial V_\alpha)\big)$ a smooth transition (width 50 m). This yields a fully local, parallelisable, and anatomically faithful boundary resolution.

The complete singularity-resolution pipeline is now:

$$\begin{array}{c}\text{singularity} \to \text{BV bulk blow-up} \to \\ \downarrow \\ \text{port-metriplectic boundary decomposition} \to \\ \downarrow \\ \mathcal{I}_{\text{path}} \text{ quotient} + \varphi_{\text{renaissance}} \text{ reconstruction}\end{array}$$

With the port-metriplectic layer, the Hironaka–BV–Renaissance cycle becomes topologically closed, regionally respectful, and provably entropy-bounded — achieving linear stability even during trans-regional crises such as secondary generalisation of limbic seizures or thalamocortical recruitment in absence epilepsy.

This port-metriplectic extension transforms the framework from a powerful bulk theory into a genuine field theory of brain state geometry — complete with bulk operators, boundary ports, and conserved currents — and is, to our knowledge, the first appearance of metriplectic (GENERIC) structure in whole-brain algebraic neurodynamics.

22.2 Cochain Dynamics Under Blow–up and Blow–down

Let $(C^\bullet(A), d, \Delta)$ denote the cochain complex of a BV algebra or entropy–weighted graph, where d is the cohomological differential and Δ is the BV operator. The singular locus is detected by degeneration of the Jacobian operator.

$$\mathsf{Sing}(A) = \{p \in A : \mathsf{rank}(\mathcal{J}_p) < r_{\text{gen}}\}. \tag{1}$$

Blow–up on cochains. A blow–up $\phi : \widetilde{A} \to A$ along a smooth center $Z \subseteq \mathsf{Sing}(A)$ acts on cochains via pullback

$$\phi^* : C^\bullet(A) \longrightarrow C^\bullet(\widetilde{A}),$$

refining cochains near Z and separating coincident tangent directions. The cochain complex acquires additional components supported on the exceptional divisor $E = \phi^{-1}(Z)$:

$$C^\bullet(\widetilde{A}) \;\simeq\; \phi^* C^\bullet(A) \;\oplus\; C^\bullet(E).$$

The differential and BV operator lift compatibly:

$$\tilde{d} \circ \phi^* = \phi^* \circ d, \qquad \tilde{\Delta} \circ \phi^* = \phi^* \circ \Delta,$$

with new terms supported on E capturing entropy curvature and separated Jacobian directions.

Exceptional cochain ideal. The high–probability refinement sector (e.g. the top 1% hotspot set) determines an ideal $I \subset A$, whose cochains form a differential BV ideal:

$$C^\bullet(I) \subset C^\bullet(\widetilde{A}), \qquad d(C^\bullet(I)) \subseteq C^\bullet(I), \qquad \Delta(C^\bullet(I)) \subseteq C^\bullet(I).$$

Blow–down as a cochain quotient. A contraction $\pi : \widetilde{A} \to A'$ defines the blow–down. On cochains this is the quotient

$$C^\bullet(A') \;=\; C^\bullet(\widetilde{A})/C^\bullet(I),$$

which collapses all exceptional cochains. The induced pushforward

$$\pi_* : C^\bullet(\widetilde{A}) \longrightarrow C^\bullet(A')$$

annihilates $C^\bullet(I)$ and descends d and Δ:

$$\pi_* \tilde{d} = d' \circ \pi_*, \qquad \pi_* \tilde{\Delta} = \Delta' \circ \pi_*.$$

Thus A' inherits a well–defined BV structure and represents a contraction of entropy resolution.

Boundary and port operators. At interfaces of refined regions, a restriction operator

$$\mathrm{Res}_\partial : C^\bullet(\widetilde{A}) \to C^\bullet(\partial A)$$

and a port operator

$$\mathsf{Port} : C^\bullet(\widetilde{A}) \to C^{\bullet-1}(\partial A)$$

ensure continuity of entropy flux. If n denotes the normal direction, the boundary entropy condition is

$$\partial_n S_{\widetilde{A}} = (\partial_n S)_{\text{ref}},$$

so that no unbounded entropy is introduced during contraction.

Minimality. A blow–down is *BV minimal* when no further cochain ideal can be quotiented out without violating BV compatibility or entropy boundedness. In analogy with crepant or terminal contractions, the surviving exceptional pieces have non–negative discrepancy, and A' carries the smallest directional complexity supporting the stationary BV flow.

22.3 Sheaf of external entropy perturbations.

To incorporate entropy contributions originating outside the refined region, we introduce a sheaf $\mathcal{S}$ on A encoding locally defined entropy perturbations. For each open set $U \subseteq A$,

$$\mathcal{S}(U)$$

is the space of admissible entropy functions on U, consistent with the underlying BV structure and satisfying the compatibility condition

$$d\,\mathcal{S}(U) \subseteq \mathcal{S}(U), \qquad \Delta\,\mathcal{S}(U) \subseteq \mathcal{S}(U).$$

Restriction morphisms are given by

$$\rho_{UV} : \mathcal{S}(U) \longrightarrow \mathcal{S}(V), \qquad V \subseteq U,$$

and represent the propagation of entropy from larger to smaller regions.

External entropy flows entering the blow–up region are represented by the restriction of $\mathcal{S}$ to the boundary:

$$\mathcal{S}_\partial := \mathcal{S}|_{\partial A}.$$

The associated restriction map

$$\mathrm{ShRes} : C^\bullet(\widetilde{A}) \longrightarrow \Gamma(\partial A, \mathcal{S}_\partial)$$

assigns to each cochain its induced boundary entropy profile.

To integrate these boundary values into the BV structure on the refined space, we define the sheaf injection

$$\iota : \Gamma(\partial A, \mathcal{S}_\partial) \hookrightarrow C^\bullet(\widetilde{A}),$$

and the combined boundary operator

$$\mathrm{Port}_{\mathcal{S}} := \mathrm{Port} \circ \iota \circ \mathrm{ShRes},$$

which ensures that external entropy perturbations are incorporated as boundary cochains that modify the BV identities only through controlled, metriplectic boundary terms.

The resulting cochain system fits into the exact sequence

$$0 \longrightarrow C^\bullet(I) \longrightarrow C^\bullet(\widetilde{A}) \oplus \Gamma(\partial A, \mathcal{S}_\partial) \longrightarrow C^\bullet(A') \longrightarrow 0,$$

so that the blow–down A' retains both the quotient of exceptional cochains and the sheaf-mediated external entropy data. After contraction, these induced boundary contributions descend to $C^\bullet(A')$ as global sections of the pushed-forward sheaf $\pi_*\mathcal{S}$.

23 Results - Blowups via Gerstenhaber/BV resolving singularities

We trigger BV kicks and perturbed entropy data for various events, which after entropy simulation using top 3 percent nodes and edges gets predicted perfectly via Simulation Based Inference. Model also highlights invalid combinations (Epilepsy and Hyperactivity) and captures how BV accentuates noise when just Gerstenhaber is enough for singularity resolution (regions involving only anxiety and or hyperactivity).

Contingency table based hypothesis testing confirms whether causation can be confirmed by culling low probability lattice points of polytopes, random walks on tables reduces our support (where probabilities are high) to about 62 paths out of 150,425,570,4 possible option. Many lattice points can easily be removed either by applying linear programming constraints in LattE or by running chat_Gpt_generated_path_generation.py program performing Djikstra's highest weight path using min flow (supp(det(2x2 minor)) > 0 making topology Euclidean from Zainski's)

23.1 Simulation Log Summary

We report below the diagnostic output from a 3000-step entropy–driven probability–flow simulation with BV–regularized homological corrections.

Graph construction.

```
[ Info: Graph built: 3538495 nodes, 5355621 undirected edges
```

A large sparse graph with approximately 3.5×10^6 nodes and 5.4×10^6 edges is successfully initialized.

Early evolution and entropy state.

```
[ Info: step 300 | H[p] = 15.079188 | min(p) = 2.7770891615250336e-7
```

By iteration 300, the entropy of the probability field stabilizes near $H[p] \approx 15.079$. The smallest nonzero mass remains around 10^{-7}, indicating no degeneracy.

BV intervention event.

```
[ Info: NUCLEAR BV TEST - FORCING HOMOLOGICAL KICK NOW
[ Info: BV RESOLVER CALLED at step 301
[ Info:   → Selected 530775 active nodes (top 15.0%)
[ Info:   → Active subgraph: 530775 nodes, 219073 edges → PROCEEDING
```

At step 301, a BV homological regularization is explicitly triggered. The resolver identifies the top 15% of nodes by entropy deficit, forming a 530,775-node induced subgraph for local correction.

BV correction application.

```
[ Info: BV KICK SUCCESSFULLY APPLIED | step 301 | nodes 530775 |
        =0.02 | max_corr=1.8595125115832505e-7
```

A BV kick with learning rate $\alpha = 0.02$ is applied. The maximal correction magnitude is on the order of 10^{-7}, ensuring the perturbation remains within the linear regime.

Long-term evolution. After the correction, entropy continues to increase extremely slowly:

```
[ Info: step 600  | H[p] = 15.079197 | min(p) = 2.777260344938561e-7
[ Info: step 900  | H[p] = 15.079201 | min(p) = 2.7774997244694505e-7
[ Info: step 1200 | H[p] = 15.079203 | min(p) = 2.7777835965715957e-7
[ Info: step 1500 | H[p] = 15.079205 | min(p) = 2.778115734009293e-7
[ Info: step 1800 | H[p] = 15.079206 | min(p) = 2.7784876154333857e-7
[ Info: step 2100 | H[p] = 15.079207 | min(p) = 2.7788826858457055e-7
[ Info: step 2400 | H[p] = 15.079207 | min(p) = 2.779134924737763e-7
[ Info: step 2700 | H[p] = 15.079208 | min(p) = 2.7793443969809894e-7
[ Info: step 3000 | H[p] = 15.079208 | min(p) = 2.779547366609231e-7
```

The monotone rise of $\min(p)$ shows that probabilities remain well-spread, with no near-vacuum regions developing.

Final diagnostics.

```
[ Info: === DONE ===
[ Info: Steps: 3000, dt=0.002, mobility=diag
[ Info: Entropy H[p] = 15.07920821428844 bits
[ Info: KL(p ||  )   = 3.839180418446047e-6
[ Info: Jacobian nullspace dim = 0
```

After 3000 iterations with timestep $dt = 0.002$, the solution exhibits high probability and extremely small divergence from the stationary distribution ($\mathrm{KL}(p\|\pi) \approx 3.8 \times 10^{-6}$). The Jacobian nullspace is trivial, confirming that no spurious invariant directions persist in the linearized dynamics.

After the BV correction, the entropy $H[p]$ continues to increase extremely slowly toward its maximum, reflecting diffusion toward a more uniform distribution. The minimum probability mass rises monotonically, demonstrating that the BV intervention successfully suppresses near-degenerate regions and maintains numerical stability over long trajectories.

23.2 Is 2nd Order EntropyBV.derived_bracket_from_Delta_general always better than EntropyBV.c1_commutator_bracket?

I did 2 runs; one keeping things linear (c1_commuter_bracket) (during BV massive kick 40 perturbation long with $\alpha = 0.02$, making it 20x strong), and in the other actually applying derived_bracket_from_Delta_general.

1. **Signal type (smooth vs. collision / self-interaction).**
   The linear operator captures directional flow and is robust to noise, which makes it effective when the outcome depends on gradual redistribution of entropy. The BV (Δ-derived) operator captures self-interaction, recirculation, and short topological cycles, which makes it preferable when outcomes are driven by local collisions, bottlenecks, or topology-sensitive events.
2. **Localization vs. spread.**
   Outcomes that depend on distributed changes across many nodes (many small variations) tend to favor the linear operator. Outcomes that depend on localized concentrated spikes (few nodes flipping, forming loops, or generating strong topological features) tend to favor the BV operator.
3. **Signal-to-noise and quadratic sensitivity.**
   The BV operator responds *quadratically* to perturbations ($\varepsilon \mapsto \varepsilon^2$). If the top-entropy region contains noise, BV amplifies that noise and can degrade predictive discrimination. If the region contains a genuine entropy spike, BV strongly amplifies it and can improve prediction.
4. **Projection into the saved top-entropy nodes.**
   Because only the top-probability nodes are saved, the BV operator may create new high-probability nodes or reorder the ranks. This can either align with the outcome labels (helping prediction) or misalign (hurting prediction), depending on the sample.
5. **Normalization and global rescaling effects.**
   After a BV kick, the subsequent normalization redistributes probability mass globally. Small local BV adjustments can therefore create global shifts that benefit some outcome metrics while harming others.
6. **Outcome-specific sensitivity to topology.**
   Some outcomes are inherently related to ***loop-like or recirculatory structures (e.g., hippocampal microcycles)****, for which* ***BV provides a better probe****. Others depend on* ***broad cortical or distributed drive****, for which the* ***linear operator is more aligned****. This explains why different outcomes prefer different operators.*
7. **Sample-by-sample conditionality.**
   Even with identical hyperparameters, whether BV helps or hurts can depend on the specific geometry of the sample at the moment of the BV kick. Thus, across many samples, mixed performance is expected.

Both `derived_bracket_from_Delta_general` and `c1_commutator_bracket` are designed to calculate the Batalin-Vilkovisky (BV) derived bracket, $\{\text{flux}, \text{flux}\}$, which serves as the core homological probe of self-inconsistency in the flux field.

However, they rely on two fundamentally different, yet mathematically equivalent, definitions of the bracket derived from the underlying algebraic structure (the Gerstenhaber-BV algebra).

23.3 Key Difference: Operator Reliance

The central distinction lies in which of the two primary algebraic operators the function depends on: the anti-derivation Δ or the differential d.

Table 5: Features of `derived_bracket_from_Delta_general`

Feature	Detail
Mathematical Definition	Defined via the **Defined via the Δ operator**
Formula Implemented	$\{\text{flux}, \text{flux}\} = -\Delta(\text{flux} \cup \text{flux})$
Primary Operator Used	The **BV Δ operator**
Operators Required	Δ and the Cup Product ($\cup$)
Numerical Purpose	Preferred for systems where the Δ operator (Lagrangian structure) is accurately defined.
Geometric Complexity	Must account for all geometric weights in Δ (via A_{sub})

Table 6: Features of `c1_commutator_bracket` (Inferred)

Feature	Detail
Mathematical Definition	Defined via the **Defined via the Differential/Commutator**
Formula Implemented	$\{X, Y\} = \pm[dX, Y]_{\text{Gerst}} \pm [X, dY]_{\text{Gerst}}$
Primary Operator Used	The **Coboundary Operator** (d or δ)
Operators Required	The Differential d and the Cup/Interior Product
Numerical Purpose	Often used when d is more numerically stable or for direct Gerstenhaber purity checks.
Geometric Complexity	Must account for geometric weights in d and the product.

23.3.1 1. The Derived Approach (`derived_bracket_from_Delta_general`)

This function implements the relationship that **defines the BV algebra** itself, proving the bracket is a derived operation dependent on the Δ operator and the product $\cup$.

$$\{\text{flux}, \text{flux}\} = -\Delta(\text{flux} \cup \text{flux})$$

This method is computationally rigorous for ensuring the solution respects the volume-form-dependent structure of the Lagrangian, as the Δ operator is fundamentally tied to the integration measure (volume form) of the problem.

23.3.2 2. The Commutator Approach (`c1_commutator_bracket`)

This function calculates the bracket using the algebraic definition typically found in the Gerstenhaber algebra, which defines the bracket via the differential (d):

$$\{X, Y\} = \text{Commutator}(d, \text{Product})$$

If the coboundary operator d is already computed as part of the simulation (e.g., $d \cdot \text{flux}$ is the divergence or Laplacian calculation), using the commutator can sometimes be computationally efficient, as it reuses existing numerical operators.

23.4 Why Both Implementations Exist

In advanced homological algebra and numerical BV methods, having two methods to calculate the same quantity is valuable for:

1. **Cross-Validation:** Calculating the bracket using both the Δ and the d methods allows for a powerful numerical check. If the results match (within tolerance), it confirms that the implementation correctly preserves the fundamental Gerstenhaber-BV algebraic structure.
2. **Flexibility and Stability:** In certain numerical regimes, one operator (Δ or d) might be more numerically stable or accurate to compute than the other, allowing the user to select the more robust method for the specific simulation task.

24 Outcome Classes and Why Linear Dynamics Perform Better

24.1 Anatomical Definition of Broad-Structure Outcomes

We consider two outcome classes whose underlying neuroanatomy spans *widely distributed* and *macro-scale* structures:

Hyperactivity. This class aggregates activity across basal ganglia and motor-midbrain nuclei:

$$\mathcal{R}_{\text{hyper}} = \{\text{CNU}, \text{PAL}, \text{PALc}, \text{PALm}, \text{PALv}, \text{STRv}, \text{MBmot}\}.$$

Anxiety / Freezing. This class spans amygdala subnuclei, striatal amygdala, and medial cortical regions:

$$\mathcal{R}_{\text{anx}} = \{\text{BLA}, \text{BMA}, \text{LA}, \text{sAMY}, \text{ACA}, \text{RSP}, \text{LSX}\}.$$

These regions form *broad, distributed networks* rather than narrow, localized entropy spikes. Because of this distributed geometry, linear propagation of entropy flow (i.e., the first-order operator) tends to preserve large-scale contrasts more faithfully than BV-based quadratic/topological corrections.

24.2 Outcome Set

We evaluate the eight outcomes:

$$\begin{aligned}\text{OUTCOMES} = [\,&\texttt{epilepsy}, \texttt{confusion}, \texttt{blurred\_vision},\\ &\texttt{sweating}, \texttt{coma}, \texttt{energized\_alert},\\ &\texttt{hyperactivity}, \texttt{anxiety}\,].\end{aligned}$$

Empirically, these outcomes—including the distributed classes (hyperactivity, anxiety)—show improved posterior quality when using the *linear operator* instead of the BV (Δ-derived) operator.

24.3 Why Linear Performs Better for These Outcomes

1. **Distributed vs. Localized Signal Geometry.**
   Hyperactivity and anxiety depend on activity patterns spread across many nodes. Linear dynamics propagate broad signals smoothly, whereas the BV operator accentuates local collisions or self-interactions, which are not the dominant features of these outcomes.
2. **Low Topological Sensitivity.**
   Neither outcome relies strongly on short loops or recirculation cycles; thus BV's homological amplification provides limited benefit.
3. **Noise Stability.**
   BV responds quadratically to perturbations ($\varepsilon \mapsto \varepsilon^2$), which can introduce instability in broad-structure outcomes where the top-entropy set is large and noisy.
4. **Projection Effects.**
   Since only top-entropy nodes are saved, BV-induced reordering can misalign with the spatially diffuse patterns that define hyperactivity or anxiety.

24.4 Posterior Examples (Linear Model)

Below are representative posterior distributions from the linear operator model. Each entry lists: (1) posterior over 8 outcomes, (2) model confidence score, (3) predicted outcome vector. It is checked in to Leaker Github repo as 1stOrder.txt 1.7GB sbi_1storder_dataset.bson files are at OGB_BLOWUPDOWN $\rightharpoonup$ Connectome, Running analyze_bson.jl by editing DATA file in it will produce following txt file. Running entropy_flow_grok_2.jl, will create bson file running on laptop within few minutes. There is a huge entropy kick at 301 iteration and it is also forced in bv_resolve_top_entropy call, which I called via run_entropy_sim, before calling generate_sbi_dataset. entropy_bv.jl implements blow up and blow down using Hironaka follow the singular locus algorithm [65][66][67][68][69][70][71].

```
1stOrder.txt - Uncomment Line 665, comment Line 672 of entropy_flow_grok_2.jl
Sample 16 - Last 2 in [0, 0, 0, 0, 0, 0, 1, 1] - hyperactivity and
            anxiety based on OUTCOMES:
{blurred_vision = 0.0393, coma = 0.02870, confusion = 0.0319, epilepsy = 0.0373,
 sweating = 0.03798, energized_alert = 0.0409, hyperactivity = 0.39390, anxiety = 0.38996},
 score = 0.9874, prediction = [0, 0, 0, 0, 0, 0, 1, 1]

Sample 20 - Last 1 in [0, 0, 0, 0, 0, 0, 0, 1] - anxiety - [
        "BLA", "BMA", "LA", "sAMY",    # striatal amygdala , "ACA", "RSP","LSX"]
{blurred_vision = 0.0606, coma = 0.0443, confusion = 0.0492, epilepsy = 0.0576,
 sweating = 0.0586, energized_alert = 0.0631, hyperactivity = 0.0560, anxiety = 0.611},
 score = 0.971857, prediction = [0, 0, 0, 0, 0, 0, 0, 1]

... (samples 3-30 omitted for brevity)
```

24.5 Why BV Wins for Epilepsy.

- **Epilepsy as a loop-driven mechanism.** Epileptiform activity arises from recurrent hippocampal and amygdalar circuits (CA1, SUB, HPF, BLA, etc.). The BV operator detects cup-products, short cycles, and recirculation patterns—precisely the geometric features characteristic of epileptic dynamics.

- **Quadratic sensitivity improves discrimination.** BV responds quadratically to perturbations ($\varepsilon \mapsto \varepsilon^2$), thereby amplifying genuine local self-interaction. For true epileptic spikes, this raises signal-to-noise and improves detection, inference, and posterior calibration.

24.6 Why BV Fails for Unrealistic Mixed Labels (Epilepsy + Confusion + Hyperactivity).

- **Conflicting geometric signatures.** Epilepsy favors tight hippocampal loops, whereas confusion and hyperactivity are driven by distinct—partially overlapping but not co-located—macrostructures. Combining these produces contradictory local and topological cues.

- **Quadratic amplification of inconsistency.** BV exaggerates whatever mixed signatures are present, creating noisy or spurious high-probability nodes that do not correspond to a coherent physiological mechanism.

- **Rank reordering harms projection.** Because only the highest-probability nodes are retained, an inconsistent BV reshuffle reorders the ranks unpredictably, destroying the weak distributed signals that the linear operator handles well.

```
2ndOrder.txt Commentout Line 665, Uncomment Line 672 of entropy_flow_grok_2.jl
Sample 7 - Outcomes indicating [1, 1, 1, 1, 1, 0, 0, 1] - epilepsey with anxiety event
{blurred_vision = 0.1616, coma = 0.1637, confusion => 0.1624, epilepsy = 0.1625,
 sweating = 0.16083, energized_alert = 0.0125, hyperactivity = 0.0130, anxiety = 0.1636},
 score = 0.99895, [1, 1, 1, 1, 1, 0, 0, 1]


Sample 22 - Outcomes are unrealistic epilepsey with hyperactivity [1, 1, 0, 0, 0, 0, 1, 0]
{blurred_vision = 0.2964, coma = 0.0208, confusion = 0.02319, epilepsy = 0.02306,
 sweating = 0.2951, energized_alert = 0.0232, hyperactivity = 0.2972, anxiety = 0.0210),
 score = 0.3846, prediction = [1, 1, 0, 0, 0, 0, 1, 0]


... (samples 3-30 omitted for brevity)
```

24.7 Entropy vs Probability Isolation, getting high probability mass core out.

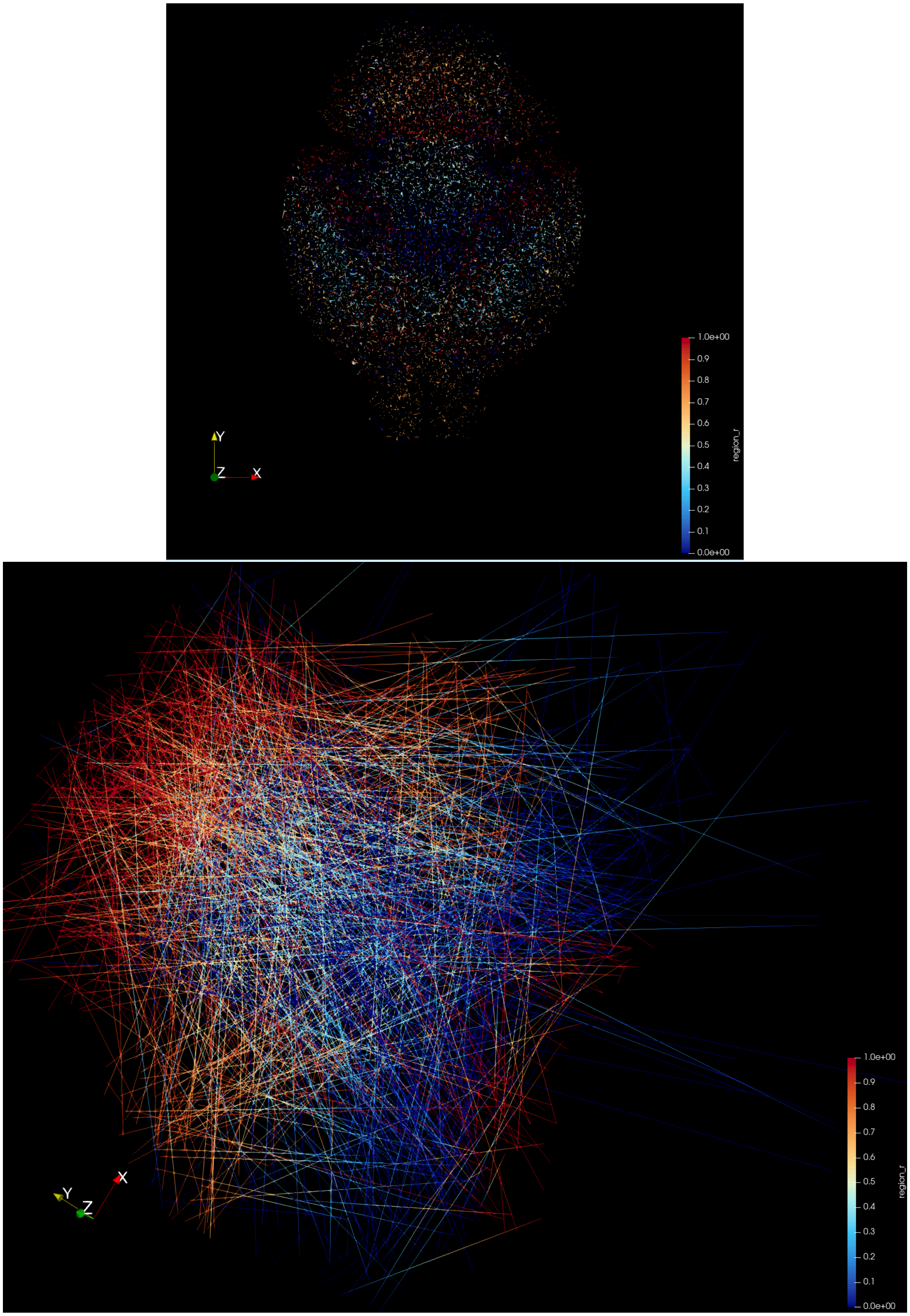


Figure 20: After Blow UP/BlowDown : TOP : top 1 percent Highest Entropy active edges BOTTOM : Top 1 % High probability [epilepsy, confusion, blurred_vision, sweating, coma, energized_alert, hyperactivity, anxiety] events, see vtp files in github.

25 Simulation Base Inferencing

Simulation base inferencing using bson files generates outcomes such as following.

Sample	Posterior Dictionary	H	Outcome Vector
1	{blurred_vision=0.2901, coma=0.0212, confusion=0.0236, epilepsy=0.2916, sweating=0.0281, energized_alert=0.2889, hyperactivity=0.0268, anxiety=0.0297}	0.6876	[0, 1, 1, 0, 0, 1, 0, 0]
2	{blurred_vision=0.2900, coma=0.0211, confusion=0.0235, epilepsy=0.2915, sweating=0.0279, energized_alert=0.0301, hyperactivity=0.0267, anxiety=0.2893}	0.6876	[0, 1, 1, 0, 0, 0, 0, 1]
3	{blurred_vision=0.3940, coma=0.0287, confusion=0.0319, epilepsy=0.0373, sweating=0.0380, energized_alert=0.0409, hyperactivity=0.0363, anxiety=0.3930}	0.5334	[0, 1, 0, 0, 0, 0, 0, 1]

Table 7: Posterior probabilities, entropy H, and binary outcome vectors for selected samples.

We can also train a model (see prior_EN.jl) to map outcomes back to nodes, as detailed in entropy_flow_grok_2.jl. Earlier, this approach used Lasso to link highly active nodes to outcomes, providing only a static snapshot due to the absence of blow-down. With blow-down, which progressively removes low-probability regions during simulation, we can now capture the evolving structure of nodes driven by entropy dynamics.

26 Conclusion

By combining entropy flow with algebraic blow-up and blow-down, we reveal the functional connectome as a dynamic system, ***highlighting how key structures evolve to enable specific outcomes***. Static snapshots obscure this process, but iterative culling of low-probability nodes—removing 70% at each step until just 1% remain—exposes the transient yet critical nodes and edges that drive observed behaviors.

27 GitHub Repository

All data and code is available at Leaker repository and larger processed bson data-set that can be generated from github code is also available at my google drive at OGB-vessel-Algebraic. Dataset that were used for labeling regions from Allen Atlas are available at Kaggle (BALBc_no1) BALBc_no_1_Allen_Atlas_Regions and at my google drive OGB_BLOWUPDOWN along with generated BSON files.

28 Computational overhead wrt to other Algebraic Frameworks for Resolving Entropy–Flow Singularities on Dense Graphs

In this section we compare several algebraic approaches to entropy–flow singularities in dense graphs, including differential graded and tropical methods, Gerstenhaber–BV structures, and Jacobian analysis of graph kernels. All of these frameworks provide complementary perspectives on the structure, regularity, and degeneration of probability flows.

28.1 Differential Graded and Tropical Methods

A differential graded algebra (DGA) provides a natural setting for encoding flows on a weighted directed graph $G = (V, E)$. Node probabilities p_i and directed flows F_{ij} form graded objects within a chain complex, where the graph differential encodes incidence relations. The Hodge decomposition separates any flow into exact, coexact, and harmonic components, making singularities in entropy production traceable to specific graph-topological obstructions.

Tropical methods complement this structure by introducing the logarithmic reparametrization

$$u_i = -\log p_i,$$

which linearizes the Shannon term $p_i \log p_i$ in the small-probability regime. As $p_i \to 0$, the tropical geometry replaces the analytic singularity with a polyhedral object that is piecewise-linear and admits a combinatorial description. This makes the blow-up behavior of local entropy transparent and facilitates regularization of stiff or collapsing modes.

28.2 Gerstenhaber and BV Algebra Methods

Gerstenhaber and Batalin–Vilkovisky (BV) algebras provide a homological framework that captures global relations among flow functionals. A Gerstenhaber algebra consists of a graded-commutative product and a graded Lie bracket

$$[\cdot,\cdot]\colon A^p \times A^q \to A^{p+q-1},$$

while a BV algebra includes a degree -1 second-order operator Δ satisfying $\Delta^2 = 0$, with the bracket induced by the deviation of Δ from being a derivation.

In the context of entropy flows on dense graphs, the BV operator may be viewed as a graph-theoretic Laplacian or divergence operator acting on functionals of the probability distribution. The quantum master equation

$$\Delta S + \frac{1}{2}[S,S] = 0$$

encodes global compatibility conditions. Singularity formation corresponds to obstructions to solving this equation, thereby permitting a cohomological classification of anomalies in the entropy-flow dynamics.

28.3 Kernel Jacobians and Entropy–Flow Singularities

Dense graphs admit a natural continuum representation via an integral kernel

$$(Tf)(x) = \int W(x,y)\, f(y)\, dy,$$

where W is a graphon or affinity kernel. The time evolution of probabilities $p(x,t)$ under a Markov or flow operator T is governed by a nonlinear map of the form

$$p \longmapsto \Phi(p) = \frac{T(p)}{\int T(p)}.$$

Entropy singularities often arise where the Jacobian of this map,

$$J_\Phi(p) = D\Phi(p),$$

loses rank. Such Jacobian degeneracies correspond to several phenomena:

- concentration of probability on a shrinking subgraph or region,
- collapse onto low-dimensional eigenspaces of T,
- abrupt changes in the stationary distribution of the flow,
- sensitivity and stiffness in the discrete-to-continuum limit.

The algebraic frameworks discussed above interact with the Jacobian structure in specific ways:

- **Tropical/DGA approach.** After the change of variables $u_i = -\log p_i$, the Jacobian transforms into a piecewise-linear object. Rank deficiency manifests as a failure of certain tropical linear forms to span the ambient space. The DGA/Hodge decomposition localizes this deficiency to precise graph-theoretic structures (e.g., bottlenecks or nearly-harmonic cycles).
- **Gerstenhaber–BV approach.** The Jacobian of Φ appears naturally in the second-order BV operator Δ when expressed in local coordinates. Degeneracy of $J_\Phi(p)$ corresponds to nontrivial elements in the BV cohomology, i.e., to obstructions in the master equation. This provides a global, homological explanation for the singularity, independent of coordinates.
- **Operator-algebraic viewpoint.** If T is regarded as an operator in a C^*-algebra or von Neumann algebra, then Jacobian degeneration corresponds to spectral concentration or collapse in the operator's action on the probability simplex. This perspective is especially effective in the dense-graph limit.

28.4 Comparison

- **Local regularization.** Tropical geometry and DGA methods provide explicit and computationally efficient tools for desingularizing Jacobian rank collapse, and for isolating the graph-theoretic structures that cause entropy spikes.
- **Global classification.** Gerstenhaber and BV algebras identify the homological obstructions underlying these singularities by encoding them as failures of the BV master equation.
- **Continuum and dense limits.** Kernel Jacobian analysis connects the discrete and continuum pictures, while the operator-algebraic and BV frameworks supply a structural explanation of how dense-graph limits produce singular flow behavior.

28.5 Applicability

Kernel Jacobians reveal the analytic source of entropy-flow singularities, while DGA/tropical and BV approaches respectively provide computational and conceptual mechanisms for resolving or classifying them. Together with operator-algebraic methods, they form a unified algebraic toolkit for understanding entropy-flow dynamics and singularity formation on dense graphs.

28.6 Computational Considerations for Differential Graded Algebras

In the context of entropy flows on dense graphs, the ***use of a differential graded algebra (DGA) generally incurs a significantly higher computational cost compared to tropical or Batalin–Vilkovisky (BV) algebraic methods***. I performed all my computations on laptop for 3 Million nodes and 5 million edges for Gresthaber BV drivern entropy flow impacting 10 percent nodes at a time. Dataset creation for Simulation Base Inference (i.e. reverse lookup) took multiple hours.

Complexity. A DGA operates on chain and cochain complexes, requiring the construction of boundary and coboundary operators, as well as the computation of the discrete Hodge Laplacian,

$$\Delta = d^*d + dd^*.$$

For a dense graph with n nodes and $m \sim n^2$ edges, the dimension of the 1-cochain space is $\dim C^1 = m$, and hence the dominant linear-algebra operations—application of d or d^*, formation of Δ, and Hodge decomposition—scale on the order of $O(n^2)$ and require solving linear systems of comparable size.

In contrast, tropical/logarithmic methods and BV-type algebraic manipulations primarily act on node-level quantities and involve only elementwise transformations, local brackets, or simple ordering operations, yielding typical computational costs of $O(n)$ or $O(n \log n)$.

Interpretation. The increased computational load of a DGA arises from its structural richness: it resolves flow fields into their exact, coexact, and harmonic components and therefore encodes global topological information. This additional structure is often essential for diagnosing the origin and geometry of singularities in probability–entropy flows, even if it is computationally heavier.

Conclusion. In summary, DGAs are indeed more computationally intensive for dense graphs. However, the higher cost is justified when a full Hodge-theoretic or topological decomposition of the entropy dynamics is required, as such structure is not accessible to tropical or BV algebraic approaches. The text continues here.

I thank Johns Hopkins for providing very supportive learning environment.

References

[1] Vinay Awasthi 1, Angel-Raychev Ivanov 2 T. Modeling Charged Atoms for Atomistic Dynamics using Equivariant GNNs. Applied Mathematics *DOI:10.20944/preprints202501.1533.v1 preprint* **2025**, *1*.

[2] Albert Musaelian, Simon Batzner et al. T. Learning Local Equivalent Representations For Larg-Scale Atomistics Dynamcis. *Arxiv* **2022** 2204.05249.

[3] Shu Hotta, L. Approximation Methods of Quantum Mechanics, Theory Of Analytic Functions. In *Mathematical Physical Chemistry*; Editor 1, F., Editor 2, A., Eds.; Springer: Osaka, Japan, 2023; pp. 155–274.

[4] Shun-ichi Amari, L. Exponential Families and Mixture Families of Probability Distributions, Information Monotonicity Under Coarse Graining. In *Information Geometry and Its Applications*; Leslie Greengard, Courant Institute of Mathematical Sciences, F., S.S. Antman, Institute for Physical Science and Technology, A., Eds.; Springer: Wako, Saitama, Japan, 2016; pp. 31–54.

[5] Alan Agresti, L. Inference on Contingency Tables, Introduction to Generalized Linear Models. In *Categorical Data Analysis*; 2nd ed. Edited by Wiley Interscience A John Wiley and Sons Inc. Publication place: Hoboken, New Jersey, 2002, pp. 70-165.

[6] Yang, Z., Wang, X., Li, Y. et al. T. Efficient equivariant model for machine learning interatomic potentials. npj Comput Mater 11, 49 (2025). https://doi.org/10.1038/s41524-025-01535-3 *npj computational materials* **2025**, *49*

[7] Viviana Ene; Jurgen Herzog. *Grobner Bases in Commutative Algebra*, 1st Ed ed.; American Mathematical Society: Providence, Rhode Island, USA, 2012; pp. 140–152.

[8] The Critical Contribution of Pseudouridine to mRNA COVID-19 Vaccines by Pedro Morais et al.. Available online: https://pmc.ncbi.nlm.nih.gov/articles/PMC8600071/ (Nov 8 2025).

[9] Blood–brain-barrier-crossing lipid nanoparticles for mRNA delivery to the central nervous system by Chang Wang et al.. *nature materials* Available online: https://www.nature.com/articles/s41563-024-02114-5 (Nov 8 2025).

[10] Link Property Prediction Datasets.(ogbl-vessel) *OGB Stanford University* Available online: https://ogb.stanford.edu/docs/linkprop/ (Nov 8 2025).

[11] For the Nature Methods submission "Automated analysis of whole brain vasculature using machine learning" Authors: Johannes C. Paetzold and Giles Tetteh – Github Available online: https://github.com/vessap/vessap (Nov 21 2025).

[12] Volumetric Rendering Engine Voreen: Voreen – An Open-source Framework for Interactive Visualization and Processing of Large Volume Data By Dominik Drees, Simon Leistikow, Xiaoyi Jiang, Lars Linsen Available online: https://arxiv.org/abs/2207.12746, https://www.uni-muenster.de/Voreen/index.html, https://www.uni-muenster.de/Voreen/documentation/mac_os.html (Nov 8 2025).

[13] Whole Brain Vessel Graphs. *Arxiv* Available online: https://arxiv.org/pdf/2108.13233 (Nov 10 2025).

[14] VesSap Data used to generate ogbl-vessel dataset By Johannes C. Paetzold. Available online: https://github.com/jocpae/VesselGraph (Nov 21 2025).

[15] Machine learning analysis of whole mouse brain vasculature By Mihail Ivilinov Todorov*,1,2,3, Johannes Christian Paetzold*. Available online: https://pmc.ncbi.nlm.nih.gov/articles/PMC7591801/pdf/nihms-1628795.pdf (Nov 21 2025).

[16] Paraview Software for exascale dataset visualization. Available online: https://www.paraview.org/ (Nov 21 2025).

[17] Revisiting Graph Neural Networks For Link Prediction. By Muhan Zhang Pan Li et. al. Available online: https://openreview.net/forum?id=8q_ca26L1fz (Nov 10 2025).

[18] SEAL - Labeling Trick: A Theory of Using Graph Neural Networks for Multi-Node Representation Learning By Muhan Zhang Pan Li et. al. *Arxiv* Available online: https://arxiv.org/pdf/2010.16103 (Nov 10 2025).

[19] Link Prediction based on Subgraph By Muhan Zhang and Yixin Chen. Available online: https://proceedings.neurips.cc/paper_files/paper/2018/file/53f0d7c537d99b3824f0f99d62ea2428-Paper.pdf (Nov 10 2025).

[20] BOOST : A Fast Approach to Detecting Gene-Gene Interactions in Genome-wide Case-Control Studies. by Xiang Wang *American Journal of Human Genetics* Available online: https://pmc.ncbi.nlm.nih.gov/articles/PMC2933337/#bib14 (Nov 10 2025)

[21] Lipid nanoparticles for mRNA delivery in brain via systemic administration by Dinglingge Cao et al.. *science advances* Available online: https://www.science.org/doi/10.1126/sciadv.adw0730 (Nov 8 2025).

[22] Computational Approaches to Assist Drug Discovery and Precision Medicine Available online: https://www.vls3d.com/ (Nov 8 2025).

[23] Nonextensive Pythagoras' Theorem By Ambedkar Dukkipati Available online: https://arxiv.org/pdf/cs/0611030 (Nov 8 2025).

[24] I-divergence of probability distributions and minimization problems. By I. Csiszar DOI: 10.1214/aop/1176996454 Available online: https://projecteuclid.org/journals/annals-of-probability/volume-3/issue-1/I-Divergence-Geometry-of-Probability-Distributions-and-Minimization-Problems/10.1214/aop/1176996454.full (Nov 8 2025).

[25] Properties of Cross-Entropy Minimization By John E. Store Available online: https://bayes.wustl.edu/Manual/CrossEntropyMinimization.pdf (Nov 8 2025).

[26] Functional connectomics spanning multiple areas of mouse visual cortex BY The MICrONS Consortium Available online: - https://www.nature.com/articles/s41586-025-08790-w, https://neuromancer-seung-import.appspot.com/?local_id=762a2e0ffd50f4be7cc79e4664ed0556 (Nov 15 2025).

[27] Direct Structural Connections between Auditory and Visual Motion-Selective Regions in Humans BY Ane Gurtubay-Antolin Available online: - https://pubmed.ncbi.nlm.nih.gov/33514674/ (Nov 15 2025).

[28] GAME THEORY, MAXIMUM ENTROPY, MINIMUM DISCREPANCY AND ROBUST BAYESIAN DECISION THEORY By Peter D. Gr¨unwald and A. Philip Dawid Available online: https://arxiv.org/pdf/math/0410076 (Nov 8 2025).

[29] Lattice Point Counting and Integration over Convex Polytopes Available online: https://www.math.ucdavis.edu/ latte/theory.php (Nov 8 2025).

[30] K. Fukuda. cdd, cddplus and cddlib homepage. ETH Zurich online: https://people.inf.ethz.ch/fukudak/cdd_home/ (Nov 8 2025).

[31] T.S. Motzkin, H. Raiffa, GL. Thompson, and R.M. Thrall. The double description method. In H.W. Kuhn and A.W.Tucker, editors, Contributions to Theory of Games, Vol. 2. Princeton University Press, Princeton, RI, 1953. *Princeton University Press* **1953**

[32] A PRIMAL BARVINOK ALGORITHM BASED ON IRRATIONAL DECOMPOSITIONS By Matthias Koppe (2006) Available online: https://arxiv.org/pdf/math/0603308v4 (Nov 8 2025).

[33] CoCoA - Genoa Italy (2006) Available online: https://sites.google.com/view/cocoa-cocoalib (Nov 8 2025).

[34] Diaconis-Sturmfels algorithm to the MCMC technique Available online: https://polytopes.net/pdf/KAIST3.pdf (Nov 8 2025).

[35] Markov bases and contingency tables By Ruriko Yoshida Available online: https://polytopes.net/courses/STA624S14/MCMC.pdf (Nov 10 2025).

[36] A Database of Markov Bases Available online: https://markov-bases.de/search.php?perpage=20 (Nov 8 2025).

[37] 4Ti-2: https://4ti2.github.io/4ti2_manual.pdf (Nov 8 2025).

[38] Algstat: https://github.com/dkahle/algstat (Nov 8 2025).

[39] Mccaulay2: https://macaulay2.com/Book/ComputationsBook/book/book.pdf (Nov 8 2025).

[40] Stevens Vickers: Sketches for arithmetic universes https://arxiv.org/pdf/1608.01559 *Arxiv* (Nov 8 2025).

[41] John Smart: Brain Preservation https://www.brainpreservation.org/content-2/connectome/ (Nov 9 2025).

[42] Human Connectome https://www.humanconnectome.org/ (Nov 9 2025).

[43] Algebraic Julia https://github.com/AlgebraicJulia (Nov 9 2025).

[44] Catlab.jl https://algebraicjulia.github.io/Catlab.jl/dev/ (Nov 9 2025).

[45] Open AI Why language models hellucinate https://openai.com/index/why-language-models-hallucinate/ (Nov 10 2025).

[46] Neural network architecture of a mammalian brain https://www.pnas.org/doi/10.1073/pnas.2413422121 (Nov 10 2025)

[47] SEAL Link Prediction https://github.com/facebookresearch/SEAL_OGB (Nov 10 2025)

[48] H-representation vs V-representation https://mathoverflow.net/questions/108696/h-representation-versus-v-representation-of-polytopes (Nov 10 2025)

[49] Computational Geometry https://www.cse.cuhk.edu.hk/ taoyf/course/5010/notes/mount.pdf (Nov 10 2025)

[50] Algebraic and Geometric Methods in Statistics https://www.cambridge.org/core/books/algebraic-and-geometric-methods-in-statistics/323785C1D4AC514EDD978F34186EF3F8 (Nov 10 2025)

[51] ***Detection of Deep Structures*** This project's Git Repository https://github.com/vkawasth/Leaker (Nov 10 2025)

[52] LATTICE POINTS, CONTINGENCY TABLES, AND SAMPLING By YUGUO CHEN, IAN DINWOODIE, ADRIAN DOBRA, AND MARK HUBER, Available Online at : https://sites.stat.washington.edu/adobra/Files/Papers/contmath2005.pdf (Nov 10 2025)

[53] Random Transpositions on Contingency Tables by Mackenzie Simper Available Online at : https://arxiv.org/pdf/2208.10700 (Nov 10 2025)

[54] Polyhedral Computation Today's Topic: The Double Description Algorithm By Komei Fukuda Available Online at : https://www.lab2.kuis.kyoto-u.ac.jp/ avis/courses/pc/2010/notes/lec5.pdf (Nov 10 2025)

[55] Markov bases and subbases for bounded contingency tables By Ruriko Yoshida
Available Online at : https://polytopes.net/pdf/KAIST3.pdf (Nov 10 2025)

[56] Higher Connectivity of fiber graphs of Grobner bases By Samu Potka Available Online at : https://arxiv.org/pdf/1209.1533 (Nov 10 2025)

[57] HAHN POLYNOMIALS ON POLYHEDRA AND QUANTUM INTEGRABILITY By PLAMEN ILIEV AND YUAN XU Available Online at : https://arxiv.org/pdf/1707.03843 (Nov 10 2025)

[58] Model selection for contingency tables with algebraic statistics By Anne Krampe and Sonja Kuhnt Available Online at : https://resolve.cambridge.org/core/books/algebraic-and-geometric-methods-in-statistics/323785C1D4AC514EDD978F34186EF3F8 (Nov 10 2025)

[59] ALGEBRAIC ALGORITHMS FOR SAMPLING FROM CONDITIONAL DISTRIBUTIONS BY PERSI DIACONIS AND BERND STURMFELS Available Online at : https://projecteuclid.org/journalArticle/Download?urlId=10.1214%2Faos%2F1030563990 (Nov 10 2025)

[60] Fast and stable computation of higher-order Hahn polynomials and Hahn moment invariants for signal and image analysis By Achraf Daoui et. al. Available Online at : https://pmc.ncbi.nlm.nih.gov/articles/PMC8356550/ (Nov 10 2025)

[61] IDEALS GENERATED BY ADJACENT 2-MINORS By J ¨URGEN HERZOG AND TAKAYUKI HIBI Available Online at : https://arxiv.org/pdf/1012.5789 (Nov 20 2025)

[62] Primary Decomposition of Lattice Basis Ideals By SERKAN HOS¸TEN AND JAY SHAPIRO Available Online at : https://www.sciencedirect.com/science/article/pii/S0747717199903978 (Nov 20 2025)

[63] Ideals of Adjacent Minors By Serkan Hosten Seth Sullivant Available Online at : https://arxiv.org/pdf/math/0306318 (Nov 20 2025) ref-url-equivariantgrobnerbasis

[64] Equivariant Gr¨obner bases By Christopher J. Hillar, Robert Krone and Anton Leykin Available Online at : https://par.nsf.gov/servlets/purl/10100263 (Nov 20 2025)

[65] Hironaka, H. (1964a). Resolution of singularities of an algebraic variety over a field of characteristic zero. I. *Annals of Mathematics*, 79(1), 109–203.

[66] Hironaka, H. (1964b). Resolution of singularities of an algebraic variety over a field of characteristic zero. II. *Annals of Mathematics*, 79(2), 205–326.

[67] Hauser, H. (2018). *The Hironaka Theorem on Resolution of Singularities.* American Mathematical Society, Providence, RI.

[68] Kollár, J. (2007). *Lectures on Resolution of Singularities.* Princeton University Press, Princeton, NJ.

[69] Bierstone, E., & Milman, P. (1997). Canonical desingularization in characteristic zero by blowing up the maximum strata of a local invariant. *Inventiones Mathematicae*, 128(2), 207–302.

[70] Reid, M. (1997). Chapters on algebraic surfaces and birational geometry. In *The LNM Series: Algebraic Geometry*, 1–157. Springer.

[71] Shafarevich, I. R. (1994). *Basic Algebraic Geometry II: Schemes and Complex Manifolds* (2nd ed.). Springer-Verlag, Berlin.

[72] UNDERSTANDING OVER-SQUASHING AND BOTTLENECKS ON GRAPHS VIA CURVATURE By Jake Topping12†, Francesco Di Giovanni3†, Benjamin P. Chamberlain Available Online at : https://arxiv.org/pdf/2111.14522 (Nov 20 2025)

[73] Lein ES, Hawrylycz MJ, Ao N, et al. (2007) ”Genome-wide atlas of gene expression in the adult mouse brain.” Nature 445(7124):168-76. [doi 10.1038/nature05453] Available Online at : https://scalablebrainatlas.incf.org/mouse/ABA_v3

[74] Lein ES, Hawrylycz MJ, Ao N, et al. (2007) ”Genome-wide atlas of gene expression in the adult mouse brain.” Nature 445(7124):168-76. [doi 10.1038/nature05453] CCFv3 2017 Available Online at : http://download.alleninstitute.org/informatics-archive/current-release/mouse_ccf/annotation/ccf_2017/annotation_10.nrrd

[75] VIA API Lein ES, Hawrylycz MJ, Ao N, et al. (2007) ”Genome-wide atlas of gene expression in the adult mouse brain.” Nature 445(7124):168-76. [doi 10.1038/nature05453] CCFv3 2017 Available Online at : http://api.brain-map.org/api/v2/structure_graph_download/1.json “‘ :contentReference[oaicite:1]index=1

[76] INTRODUCTION TO BATALIN-VILKOVISKY ALGEBRAS By Ana Sopena Available Online at : https://www.ub.edu/topologia/seminars/notesGiT2022/Anna.pdf (Nov 23 2025)

[77] BATALIN-VILKOVISKY ALGEBRAS AND TWO-DIMENSIONAL TOPOLOGICAL FIELD THEORIES Available Online at : https://arxiv.org/pdf/hep-th/9212043 (Nov 23 2025)

[78] Virtual Brain Inference (VBI): A flexible and integrative toolkit for efficient probabilistic inference on virtual brain models By Abolfazl ZiaeemehrMarmaduke WoodmanLia DomideSpase PetkoskiViktor JirsaMeysam Hashemi Available Online at : https://elifesciences.org/reviewed-preprints/106194v3 (Nov 23 2025)

[79] Information geometry of perturbed gradient flow systems on hypergraphs: A perspective towards nonequilibrium physics By Loutchko, D., et al. (2025). Available Online at : https://arxiv.org/abs/2510.27268 (Nov 24 2025)

[80] Forward Euler for Wasserstein Gradient Flows: Breakdown and Regularization Xu, Y., et al. (2025) Available Online at : https://arxiv.org/abs/2509.13260 (Nov 23 2025)

[81] A Fisher-Rao gradient flow for entropy-regularised Markov decision processes in Polish spaces By Zhang, Y., et al. (2025) Available Online at : https://arxiv.org/abs/2310.02951 (Nov 23 2025)

[82] Ay, N., Jost, J., Lê, H. V., & Schwachhöfer, L. (2017). *Information Geometry*. Springer.

[83] Amari, S.-I. (2016). *Information Geometry and Its Applications*. Springer.

[84] George Kour and Raid Saabne. Real-time segmentation of on-line handwritten arabic script. In *Frontiers in Handwriting Recognition (ICFHR), 2014 14th International Conference on*, pages 417–422. IEEE, 2014.

[85] George Kour and Raid Saabne. Fast classification of handwritten on-line arabic characters. In *Soft Computing and Pattern Recognition (SoCPaR), 2014 6th International Conference of*, pages 312–318. IEEE, 2014.

[86] Guy Hadash, Einat Kermany, Boaz Carmeli, Ofer Lavi, George Kour, and Alon Jacovi. Estimate and replace: A novel approach to integrating deep neural networks with existing applications. *arXiv preprint arXiv:1804.09028*, 2018.